



Train track fastener defect detection algorithm based on MGSF-YOLO

Siwei Ma¹ · Ronghua Li² · Henan Hu¹

Accepted: 1 February 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

The detection of train track fasteners is an important task to ensure the safe, reliable, and long-term stable operation of the railway system. Under the high stress and dynamic loads during train operation, track fasteners play a crucial role in maintaining the alignment and structural integrity of the tracks. However, traditional manual inspection methods are time-consuming, labor-intensive, and error-prone, while existing defect-detection algorithms often have the problem of low accuracy. To address these challenges, an improved YOLOv8n algorithm named MGSF-YOLO for high-precision detection of train track fastener defects is proposed. Firstly, an attention mechanism MSAM that combines MSCA and CBAM is proposed. MSAM adaptively adjusts the feature dimensions of different channels and the attention intensity at spatial positions through convolution kernels of various scales. This allows the model to focus on key features and regions and simultaneously enhances information extraction by exploiting the correlations between channels, thereby improving the model's detection performance. Secondly, GSConv is used to replace Conv. This not only reduces the number of model parameters and computational cost but also improves the model's detection accuracy. Then, an SPPFPool module that integrates average pooling and max pooling is proposed. This module aggregates the edge, background, and contextual information of the entire image, alleviates the impact of image scale changes, enhances the model's adaptability to inputs of different sizes, effectively reduces false positives and false negatives, and improves the detection accuracy. Finally, the FocalerIoU loss function is introduced to replace the original CIoU loss function, which improves the regression accuracy and accelerates the model's convergence speed. Experimental results show that compared with YOLOv8n, MGSF-YOLO has achieved performance improvements. The number of model parameters has decreased by 12.2%, and the computational amount has been reduced by 6.1%. Without sacrificing the detection speed, the mean average precision (mAP) has also increased by 2.6%. On the coco128 dataset, the publicly available NEU-DET dataset, and the real-world fastener dataset collected from the

Ronghua Li and Henan Hu have been contributed equally to this work.

Extended author information available on the last page of the article

Published online: 14 February 2025

Springer

Roboflow website, the mAP of MGSF-YOLO has increased by 1.6%, 2.4%, and 3.6%, respectively, compared to YOLOv8n. Compared with other models, MGSF-YOLO reaches the highest mAP value, fully demonstrating its superior generalization ability and detection accuracy. It provides a reliable solution for the detection of railway fastener defects.

Keywords Train track fastener defects · MSAM attention mechanism · GSConv · SPPFPool · FocalerIoU

1 Introduction

In modern railroad systems, the safe operation of trains depends on the integrity and reliability of the track and its components. As the key component connecting the track to the sleepers, the condition of fasteners directly affects the safety and efficiency of the entire railroad system. Any defects in fasteners, such as cracks, breakage, positional offset or corrosion, may lead to failure of the track structure and increase the risk of accidents [1]. Therefore, timely detection and repair of fastener defects is a critical part of maintaining railroad safety.

Traditional manual inspection methods have gradually been unable to meet the needs of modern railway systems due to high labor intensity, low efficiency and easy to be affected by subjective factors. Therefore, the development of advanced detection technology to accurately identify the state of rail fasteners has become the research focus of scholars at home and abroad [2]. With the development of machine learning and deep learning technology, many scholars have applied it to the field of fastener detection and achieved remarkable results. These methods can be mainly divided into two categories: one is traditional methods (machine learning methods), and the other is deep learning methods.

Most of the traditional methods use image processing and machine processing learning algorithms to realize the recognition of fasteners. Hu et al. [3] proposed a Bayesian model update method combined with the Python-Abaqus interface framework to identify the partial loosening of ballastless track fasteners through vibration data. Li et al. [4] adopted a method combining convolutional autoencoder (CAE) network with pseudo-Hilbert scanning to identify the loosening features of subway fasteners from the distributed vibration response of the subway track bed, effectively improving the accuracy and efficiency of extracting and identifying the loosening features of subway fasteners from complex vibration data. Liu et al. [5] proposed a novel vision-based fastener inspection system (VFIS) for automatic inspection of railway fasteners. VFIS uses an online classification method based on template matching to automatically collect and label large-scale fastener samples with only a small number of labeled fastener templates. At the same time, a similarity-based deep network is used to deal with the problem of dataset imbalance. Wang et al. [6] proposed a fastener recognition method based on improved Local Binary Pattern (LBP) operator, which can adapt to different weather conditions, detect and identify rail fastener, and show

significant improvement in detection stability. There are many problems in traditional fastener detection methods: poor applicability, insufficient robustness and easy to be affected by complex environments; It is usually necessary to rely on strict prior knowledge of orbit geometry to assist positioning, but the accuracy is low due to many restrictions. In addition, this kind of methods often require multiple algorithms to cooperate, which leads to a large amount of computation and slow localization and recognition speed. Therefore, relying on machine learning algorithms alone is not suitable to solve the fastener detection problem.

The method of deep learning mainly combines image processing and neural networks to complete the fastener detection task. Bai et al. [7] proposed a two-stage classification model based on improved Faster R-CNN and SVDD algorithm for accurate positioning and state detection of rail fasteners, so as to meet the requirements of intelligent, fast and accurate detection. As a two-stage detection algorithm based on anchor box, Faster R-CNN has a complex structure, which leads to slow positioning speed and is difficult to meet the needs of real-time detection. Ge et al. [8] proposed a method for fine-grained defect detection of rail fasteners using RGB-D images, which is divided into two stages: coarse-grained detection and fine-grained detection. The coarse-grained inspection is able to identify defects such as missing or loose elastic strips, while the fine-grained inspection identifies potential subtle defects such as loose fasteners. Combining these two detection methods, a better detection effect for rail fastener defects is achieved. However, the two detections of coarse-grained and fine-grained increase the overall computational complexity and limit the real-time performance of the algorithm. Aydin et al. [9] adopted the image enhancement technology, first filtered the railway image to obtain a high-contrast image, and then used an improved lightweight Convolutional Neural Network (LCNN) model. The model extracts features from two fully connected layers and concatenates these feature vectors, which are subsequently processed by multiple machine learning methods to select the optimal processing algorithm. However, the model feature splicing and the subsequent processing of multiple machine learning methods increase the computational complexity, and the detection speed is slow, which is not conducive to real-time performance. Su et al. [10] proposed a fast and accurate semantic segmentation network for track fastens named RFS-Net, which combined shaft-guided two-flow structure and shape attention mechanism to significantly improve the speed and accuracy of semantic segmentation for track fastens in complex environments. However, this model has strong dependence on the quality and shape characteristics of the data and shows some instability in ensuring high detection accuracy. In summary, these methods have the problems of complex calculation, slow detection speed, and unstable detection accuracy.

To solve these problems, a train track fastener defect detection algorithm, MGSF-YOLO(MSAM-GSConv-SPPFPool-FocalerIoU-You Only Look Once), based on an improved YOLOv8n(You Only Look Once version 8n), is proposed. The proposed method consists of the following steps:

- (1) The MSAM (multi-scale attention module), which combines the MSCA (multi-scale convolutional attention) and CBAM attention mechanisms, adaptively

adjusts the feature dimensions of different channels and the attention strength at different spatial locations using convolutional kernels of various scales. This enhances the model's ability to capture defect features while reducing background noise interference, thereby improving detection accuracy.

- (2) GSConv (group shuffle convolution) was used to replace Conv to reduce the number of parameters and calculation of the model, reduce the burden of the network, and improve the efficiency and speed of data processing.
- (3) The SPPFPool module (SPPF-average pooling and max pooling) is introduced, which combines the SPPF, average pooling, and max pooling methods. This method can effectively fuse the edge, background, and context information of the image, improve the model's ability to handle images of different sizes, reduce the impact of image size differences on model performance, and further improve the detection effect.
- (4) The FocalerIoU loss function was used to overcome the limitations of the existing loss functions, optimize the accuracy of regression analysis, and accelerate the convergence speed of the model.

2 Related theories

2.1 YOLOv8n structure

The YOLOv8 [11] model has five different size versions, n , s , m , l , and x . YOLOv8n is the lightest model in the YOLOv8 family and is designed for fast object detection. For the size of the dataset under study, the YOLOv8n network was chosen as the subject of this study.

The backbone network consists of Conv, C2f and SPPF, these components cooperate to extract rich features such as shape and texture of objects as well as deeper semantic information. The neck network, on the other hand, employs a feature pyramid network (PAN-FPN), a structure that effectively integrates deep and shallow feature information to improve the model's ability to recognize objects of different sizes. The neck also embeds a C2f module, which helps to expand the network's sensory field to better capture targets at all scales. For the head design of the model, YOLOv8n treats the classification head and detection head separately, which improves the model's focus and performance. For the loss function, the model uses VFLoss to deal with category imbalance, while for the regression loss, DFLLoss and CIoU Loss are combined to optimize the model and ensure that it can quickly and accurately determine and calibrate the location of objects.

The structure of YOLOv8n is shown in Fig. 1.

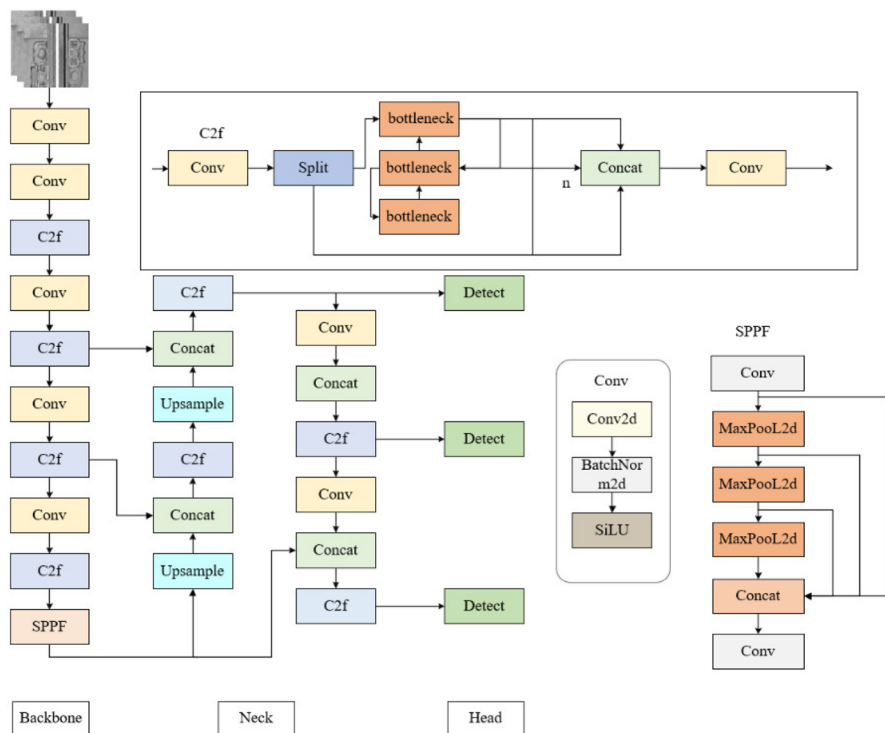


Fig. 1 YOLOv8n structure

2.2 Attention mechanism

2.2.1 CBAM attention mechanism

CBAM [12] (convolutional block attention module) attention mechanism introduces channel attention module (CAM) and spatial attention module (SAM), whose structure is shown in Fig. 2. The channel attention module focuses on the weight of the channel dimension, dynamically adjusts the importance of different channels, highlights useful features and suppresses irrelevant features. The spatial attention module dynamically adjusts the importance of different spatial locations

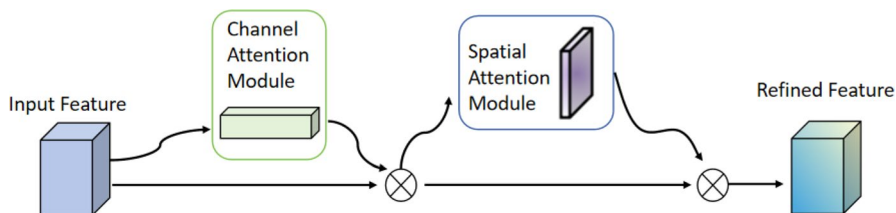


Fig. 2 CBAM structure

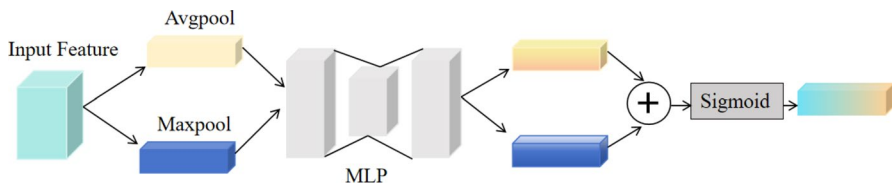


Fig. 3 CAM structure

by learning spatial weights, so as to highlight regions with important information in the image.

Since each channel in the feature map is regarded as a feature detector, the channel attention module mainly focuses on the main features of the input data. The operation process of CAM is shown in Fig. 3.

Firstly, the channel attention module was used to perform global average pooling and global maximum pooling on the input feature F , and the feature maps generated by pooling were sent to the multi-layer perceptron (MLP). The feature vectors output by MLP were added and the weight of each channel was obtained by Sigmoid function (between 0 and 1). Subsequently, the obtained weights are multiplied with the original input features to obtain the channel attention feature map $M_c(F)$, and the expression is as follows.

$$\begin{aligned} M_c(F) &= \sigma \left(\text{MLP}(\text{Pool}_{\text{Avg}}(F)) + \text{MLP}(\text{Pool}_{\text{Max}}(F)) \right) \\ &= \sigma \left(W_1 \left(W_0 \left(F_{\text{Avg}}^c \right) \right) + W_1 \left(W_0 \left(F_{\text{Max}}^c \right) \right) \right) \end{aligned} \quad (1)$$

where σ is the Sigmoid function; W_0 and W_1 denote the parameters of the multilayer perceptron. F_{Avg}^c and F_{Max}^c represent the outputs of the average pooling layer and the max pooling layer of the feature maps in the channel attention module, respectively.

The spatial attention module focuses on the importance of different regions in the feature map, which is complementary to the channel attention module, and the SAM operation process is shown in Fig. 4.

Multiply the channel attention feature map $M_c(F)$ with the feature map F to obtain F' . Global max pooling and average pooling are applied to F' in turn, respectively, to

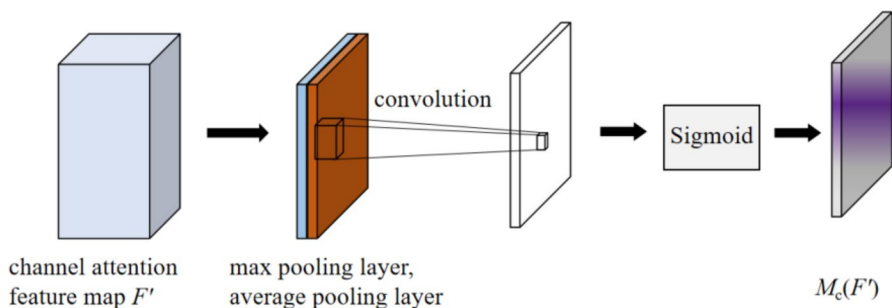


Fig. 4 SAM structure

generate feature maps. The feature maps are first concatenated in the channel dimension, and then the convolution operation is used to reduce the dimension. Finally, Sigmoid is used to activate the feature point weights (between 0 and 1) and multiply with the feature map F processed by the channel attention mechanism to obtain the spatial attention feature map $M_c(F')$. The expression is as follows.

$$\begin{aligned} M_c(F') &= \sigma(f^{7 \times 7}([Pool_{Avg}(F'); Pool_{Max}(F')])) \\ &= \sigma(f^{7 \times 7}[F'_{avg}; F'_{max}]) \end{aligned} \quad (2)$$

where: σ is the activation function, $f^{7 \times 7}$ is the alter size 7×7 convolution operation.

2.2.2 CA attention mechanism

The CA [13] (coordinate attention) attention mechanism carries out global average pooling of the input feature map in the X and Y directions, extracts the features containing coordinate information, and then generates horizontal and vertical attention maps through the convolutional layer. These attention weights are eventually multiplied with the original feature map to highlight important feature areas and channels. This mechanism can effectively capture long-distance dependencies across channels and spaces, improving model performance in various visual tasks.

CA attention mechanism averages the input feature maps of $C \times H \times W$ shape channel by channel and encodes each channel using pooling kernels of $(H, 1)$ and $(1, W)$ in X and Y axis directions, respectively, to produce feature maps of $C \times H \times 1$ and $C \times 1 \times W$ shapes. The generated feature maps are transformed and then concat operation is performed. The formula is shown in (3).

$$f = \sigma(F_1([z^h, z^w])) \quad (3)$$

The z^h, z^w are subjected to concat operation with F1 operation (dimensionality reduction using 1×1 convolution kernel) and activation operation to generate the feature map $f \in R^{C/r \times (H+W) \times 1}$. Along the spatial dimensions, f is then subjected to split operation to divide into $f^h \in R^{C/r \times H \times 1}$ and $f^w \in R^{C/r \times 1 \times W}$, and then the dimensionality enhancement operation is carried out by using the 1×1 convolution, respectively, and then the combined with sigmoid activation function to get the final attention vectors $g^h \in R^{C \times H \times 1}$ and $g^w \in R^{C \times 1 \times W}$. The output formula of CA is shown in (4), and the structure is shown in Fig. 5.

$$y_c(i, j) = x_c(i, j) \times g_c^h(i) \times g_c^w(j) \quad (4)$$

2.2.3 GAM attention mechanism

GAM [14] (global attention mechanism) and CBAM attention mechanism also use channel attention mechanism and spatial attention mechanism, but they differ in the processing of channel attention and spatial attention. The GAM

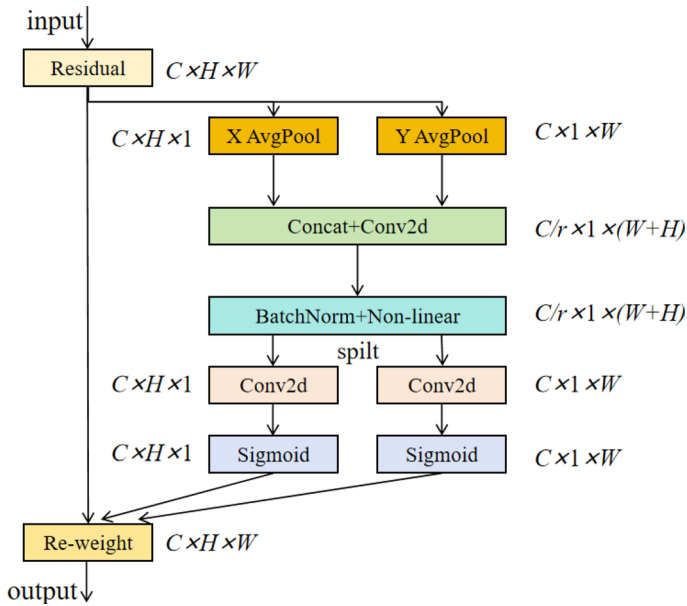


Fig. 5 CA structure

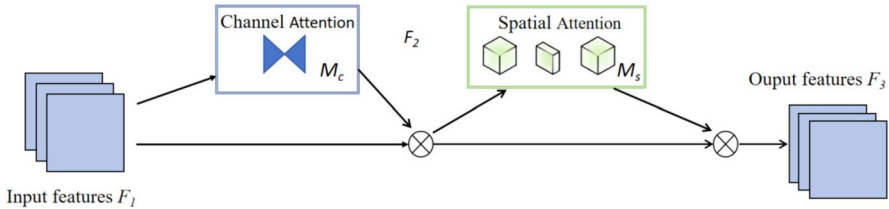


Fig. 6 GAM structure

channel attention submodule uses a 3D permutation to preserve information in three dimensions. Then, it uses a two-layer MLP (multilayer perceptron) to amplify the channel-spatial dependence across dimensions. In the spatial attention sub-module, in order to focus on spatial information, two convolutional layers are used for spatial information fusion. Given the input feature map, the intermediate state and output are defined as Eqs. (5) and (6), and the GAM structure is shown in Fig. 6.

$$F_2 = M_c(F_1) \otimes F_1 \quad (5)$$

$$F_3 = M_s(F_2) \otimes F_2 \quad (6)$$

2.2.4 SimAM attention mechanism

The SimAM [15] (similarity-based attention mechanism) attention module uses an energy function to determine the importance of each neuron in a neural network, which is defined in formula (7).

$$e = \frac{4(\partial^2 + \lambda)}{(t - \mu)^2 + 2\partial^2 + 2\lambda} \quad (7)$$

The variable t in Eq. represents a specific target neuron, μ and ∂ are calculated based on data from all other neurons than the target neuron, and λ is a hyperparameter to adjust the sensitivity of the energy function. When the energy value of the target neuron t is low, it indicates a large difference in its properties from the other neurons, and its importance is expressed in the form of $1/e$. The SimAM module weights the neuron importance as shown in Eq. (8).

$$\hat{S} = \text{sigmoid}\left(\frac{1}{E}\right) \odot S \quad (8)$$

where E represents the sum of individual energy function values in the channel and spatial dimensions of the entire network. A sigmoid activation function is used for constraints to deal with excessively large values in E and to reinforce the relative importance differences between neurons. S , $\hat{S}$ are the input feature maps and the enhanced feature maps, respectively. The SimAM structure is shown in Fig. 7.

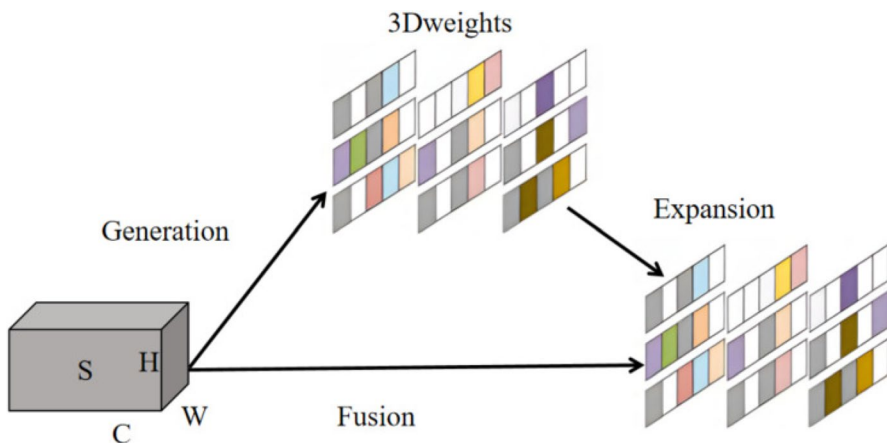


Fig. 7 SimAM structure

2.3 Convolution module

2.3.1 DCNv3

The traditional convolution operation uses a fixed convolution kernel input feature for convolution operation, and its formula is shown in (9).

$$y(p_0) = \sum_{p_n \in \mathcal{R}} w(p_n) \cdot x(p_0 + p_n) \quad (9)$$

where $y(p_0)$ is the value of the output feature map at position p_0 , $x(p_0 + p_n)$ is the value of the input feature map at position $(p_0 + p_n)$, and $w(p_n)$ is the convolutional kernel weight, $\mathcal{R}$ representing the prescribed sampling range of the convolutional kernel.

Deformable convolution, on the other hand, dynamically adapts the convolution kernel to the shape and position of the input features to better capture the spatial relationships between features. It is a convolution operation that can adapt itself to the deformation of the input features by learning and consists of an offset prediction network and a deformable convolution operation. It first computes the sample points at each input location and then computes the output features based on the sample points and offsets.

DCNv3 [16] (deformable convolutional networks version 3) builds on DCNv2 by introducing a multi-grouping mechanism to share weights among convolutional neurons as well as normalizing the modulation scalar among sampling points. The DCNv3 operator serves as an extension of the DCN sequence with higher computational and memory efficiency based on sparse sampling. The formula is given in (10).

$$y(p_0) = \sum_{p_n \in \mathcal{R}} w(p_n) \cdot x(p_0 + p_n + \Delta p_n) \quad (10)$$

Here, Δp_n represents the offset, which is learned through an additional network layer that allows the convolutional kernel to adaptively adjust the sampling position to better align the local structure of the input features.

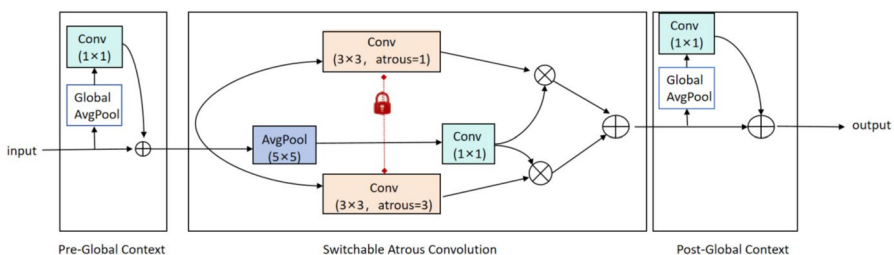


Fig. 8 SAConv structure

2.3.2 SAConv

The architecture of SAConv[17] (switchable adaptive convolution) consists of three main parts: two global context modules are located before and after the SAC component. These modules contribute to a more comprehensive understanding of the image content and enable the SAC component to work effectively in a broader context. SAC is able to convert a conventional convolutional layer into a SAC layer by using the same weights (except for a trainable difference) across convolutional operations with different hole rates. This conversion mechanism consists of an average pooling layer and a 1×1 convolutional layer to enable the on-off functionality. The SAConv structure is shown in Fig. 8.

2.3.3 SCConv

SCConv [18] (spatial and channel reconstruction convolution) utilizes spatial and channel redundancy between features for CNN compression to reduce redundant computations and facilitate the learning of representative features. SCConv is shown in Fig. 9, which consists of two units, a spatial reconstruction unit (SRU) and a channel reconstruction unit (CRU) placed in a sequential manner. Specifically, for the intermediate input feature X in the bottleneck residual block, the spatial refinement feature X^w is first obtained by SRU operation, and then the channel refinement feature Y is obtained by using CRU operation. The SCConv module takes full advantage of the spatial redundancy and the channel redundancy among the features, and it can be seamlessly integrated into any CNN architecture to reduce the redundancy among the intermediate feature mappings and to enhance the feature representation of CNNs.

2.4 Spatial pyramid pooling module

2.4.1 SimSPPF

SimSPPF [19] (simplified spatial pyramid pooling-fast) allows the network to capture spatial information at different scales, and the pooled feature maps are spliced with the output of another convolutional layer in the channel dimension. This feature fusion brings in rich contextual information, which not only

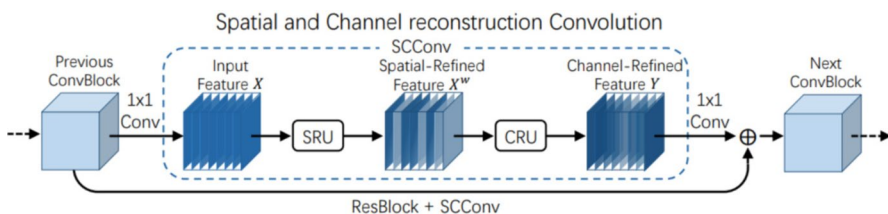


Fig. 9 SCConv structure

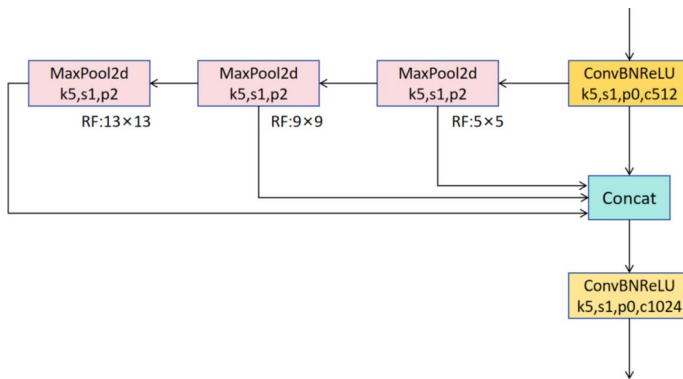


Fig. 10 SimSPPF structure

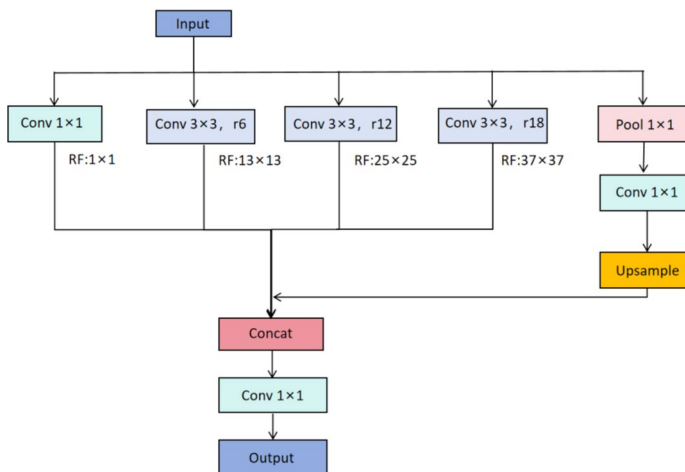


Fig. 11 ASPP structure

extracts rich multi-scale features, but also maintains the ability to perceive different object scales. The structure of SimSPPF is shown in Fig. 10.

2.4.2 ASPP

The ASPP [20] (atrous spatial pyramid pooling) module uses multiple parallel null convolutional layers with different sampling rates, and the features extracted for each sampling rate are further processed in separate branches and fused to generate the final result. The module constructs convolutional kernels with different receptive fields by different null rates, which are used to obtain multi-scale object information, and the specific structure is shown in Fig. 11.

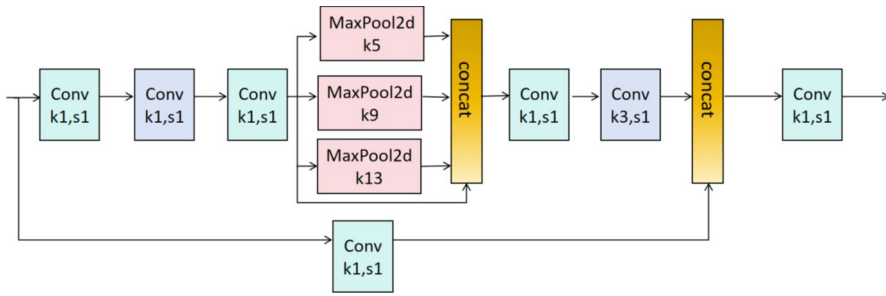


Fig. 12 SPPCSPC structure

2.4.3 SPPCSPC

The SPPCSPC [21] (spatial pyramid pooling cross stage partial channel) module first performs pooling of multiple dimensions on the input feature maps using pooling operations at different scales, respectively, to fuse these pooled feature maps together to form a multi-scale feature map. Then a 1×1 channel-by-channel convolution operation is utilized to achieve the integration of the multi-scale feature maps in the channel dimension to further extract feature information. The SPPCSPC structure is shown in Fig. 12.

2.4.4 SPELAN

SPPELAN [22] (spatial pyramid pooling efficient layer aggregation network) input X first passes through a transformation layer for initial feature extraction, after which the data stream is segmented and simultaneously passed through a convolutional layer and three different maximal pooling layers, in order to extract features from

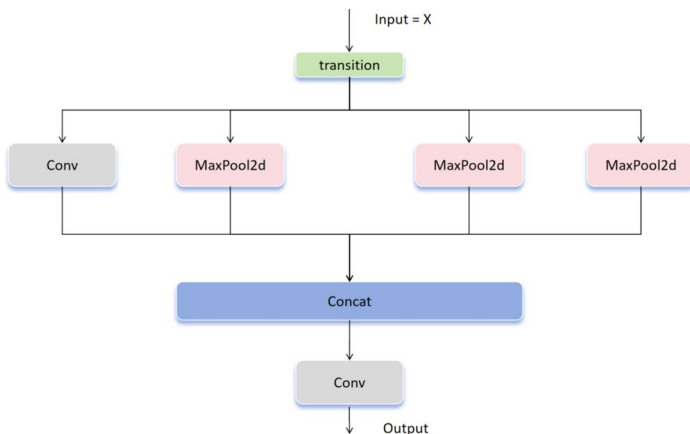


Fig. 13 SPELAN structure

different sizes of the sensory field. Then, all the feature maps are spliced together to merge the multi-scale information. The spliced feature maps are processed through another convolutional layer to further synthesize the features and get the final output. The SPPELAN structure is shown in Fig. 13.

2.5 Loss function

2.5.1 EIoU loss function

EIoU [23] (efficient-IoU) consists of three components: the IoU loss, the distance loss, and the height and width loss. The height and width loss directly minimizes the difference between the height and width of the predicted target bounding box and the real bounding box, resulting in faster convergence and better localization results.

The EIoU calculation formula is shown in (11).

$$\begin{aligned} L_{\text{EIoU}} &= L_{\text{IoU}} + L_{\text{dis}} + L_{\text{asp}} \\ &= 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{\text{gt}})}{c^2} + \frac{\rho^2(w, w^{\text{gt}})}{C_w^2} + \frac{\rho^2(h, h^{\text{gt}})}{C_h^2} \end{aligned} \quad (11)$$

where C_w and C_h are the width and height of the minimum outer rectangle of the predicted bounding box from the true bounding box. ρ is the Euclidean distance between the two points.

2.5.2 SIoU loss function

The SIoU [24] (SCYLLA-IoU) loss function is mainly composed of four parts such as Angle cost, Distance cost, Shape cost and IoU cost, which are calculated as follows. The Angle cost section is defined in the following form:

$$A = 1 - 2\sin^2\left(\arcsin\left(\frac{c_h}{\sigma}\right) - \frac{\pi}{4}\right) \quad (12)$$

where c_h is the height difference between the center point of the real frame and the predicted frame, σ is the distance between the center point of the real frame and the predicted frame.

The Distance cost section is defined in the following form:

$$\Delta = \sum_{t=x,y} (1 - e^{-\gamma\rho_t}) = 2 - e^{-\gamma\rho_x} - e^{-\gamma\rho_y} \quad (13)$$

$$\rho_x = \left(\frac{b_{c_x}^{\text{gt}} - b_{c_x}}{c_w}\right)^2, \rho_y = \left(\frac{b_{c_y}^{\text{gt}} - b_{c_y}}{c_h}\right)^2 \quad (14)$$

$$\gamma = 2 - A \quad (15)$$

where $(b_{c_x}^{gt}, b_{c_y}^{gt})$ is the center coordinate of the real frame, (b_{c_x}, b_{c_y}) is the center coordinate of the prediction frame, c_w, c_h is the width and height of the smallest outer rectangle of the real frame and the prediction frame.

The Shape cost section is defined in the following form:

$$\Omega = \sum_{t=w,h} (1 - e^{-w_t})^\theta = (1 - e^{-w_w})^\theta + (1 - e^{-w_h})^\theta \quad (16)$$

$$w_w = \frac{|w - w^{gt}|}{\max(w, w^{gt})}, w_h = \frac{|h - h^{gt}|}{\max(h, h^{gt})} \quad (17)$$

where (w, h) and (w^{gt}, h^{gt}) is the width and height of the predicted and real frames, respectively, and θ controls the amount of attention paid to shape loss.

The IoU cost section is defined in the following form:

$$L_{IoU_Cost} = 1 - \text{IoU} \quad (18)$$

In summary, the composition of the formula for the entire SIOU loss function is shown in (19).

$$\text{SIOU} = 1 - \text{IoU} + \frac{\Delta + \Omega}{2} \quad (19)$$

3 Technical approaches

3.1 MGSF-YOLO structure

The MGSF-YOLO model proposed in this paper optimizes the structure of the original YOLOv8n model, and the improved model performs better than the previous model in object detection and also realizes the balance between model lightweight and performance. And it is verified on multiple datasets, and the network structure diagram of MGSF-YOLO is shown in Fig. 14. MGSF-YOLO is mainly composed of three components: Backbone, Neck, Head. Firstly, the designed MSAM attention mechanism is introduced at the tail of the Backbone network to capture the channel features at different scales to improve the model's ability to identify defects and improve the generalization performance. Secondly, the GSConv module was used to replace the Conv module. This method reduced the number of parameters and computational complexity of the model, while maintaining high feature expression ability and detection performance. Then, the SPPFPool module was proposed by fusing average pooling and max pooling with SPPF, which realized the complementarity of salient features and global information under multi-scale feature extraction. Max pooling focuses on extracting the key features of the target and is robust to noise, while average pooling retains the global background information and reflects the overall distribution of the features. Through the fusion of the two pooling methods, SPPFPool can enrich feature expression, reduce information loss, and enhance the

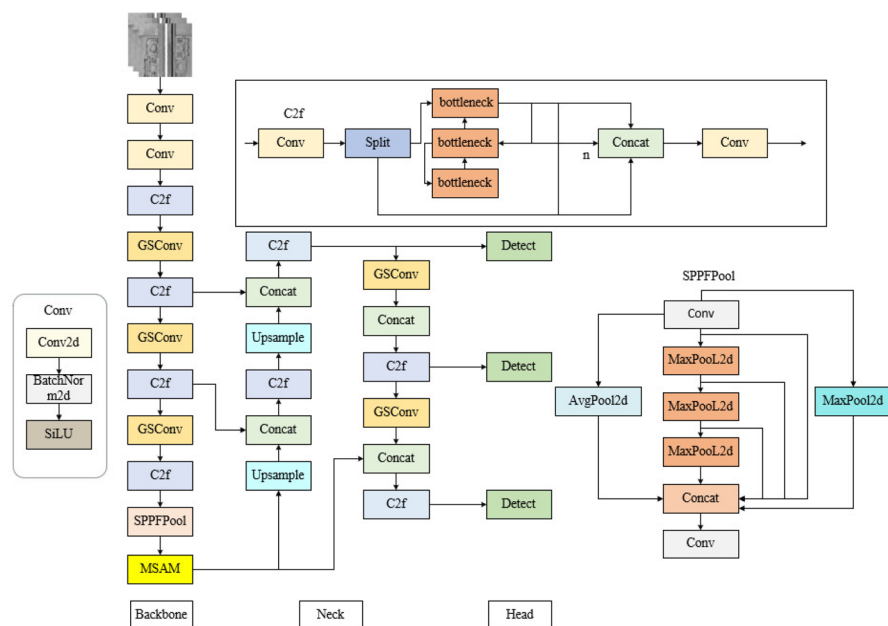


Fig. 14 MGSF-YOLO structure

stability of the model in complex scenes. Finally, the FocalerIoU loss function is used to replace the CIoU loss function to improve the positioning accuracy of the bounding box, avoid wasting too much computing resources, so as to focus the target faster and effectively improve the convergence speed of the model.

3.2 MSAM attention mechanism

CBAM combines channel attention and spatial attention, but it imposes a consistent spatial attention distribution across all channels of the output feature, which limits its flexibility in selecting important regions. Therefore, this paper introduces the MSAM attention mechanism, which adapts the attention strength across different channels in both the feature dimension and spatial locations using convolution kernels of different scales. This enhances the model's ability to capture defect features while reducing interference from background noise, thereby improving detection accuracy.

MSAM replaces the channel attention module in CBAM with MSCA [25]. The introduction of MSCA enables the model to effectively capture and utilize multi-scale information from the input features, thereby improving the model's accuracy when handling objects of different scales and complexities. By generating and fusing multi-dimensional channel weights, MSCA enhances the expressive power of the feature map, allowing the model to focus better on key channel information, improving overall performance and enhancing the model's robustness.

MSCA contains three parts: depthwise separable convolution (to capture local information), multi-branch depthwise separable convolution (to capture multi-scale context), and 1×1 convolution (to model the relationship between different channels). The output of the 1×1 convolution is directly used as the attention weight and re-used as the output weight of MSCA. MSCA can be described as follows.

$$M = \text{Conv}_{5 \times 5}(\text{Input}) \quad (20)$$

$$M_1 = \text{DWConv}_{7 \times 1}(\text{DWConv}_{1 \times 7}(M)) \quad (21)$$

$$M_2 = \text{DWConv}_{11 \times 1}(\text{DWConv}_{1 \times 11}(M)) \quad (22)$$

$$M_3 = \text{DWConv}_{21 \times 1}(\text{DWConv}_{1 \times 21}(M)) \quad (23)$$

$$\text{Output} = \text{Conv}_{1 \times 1} \left(M + \sum_{i=1}^3 M_i \right) \otimes \text{Input} \quad (24)$$

Here, $\text{DWConv}_{1 \times i}$ and $\text{DWConv}_{i \times 1}$ denote depthwise separable convolutions. M_1 , M_2 , and M_3 represent three branches, each of which uses convolution kernels of different sizes to convolutional process the input. In each branch, two depth-wise separable convolutions are used to approximate the standard depth-wise convolutions with large kernels, where the kernel sizes of three branches are set to 7, 11, 21, respectively. Finally, the results of M and $\sum_{i=1}^3 M_i$ are processed by a 1×1 convolution kernel to establish the connection between different channels. The processing result is

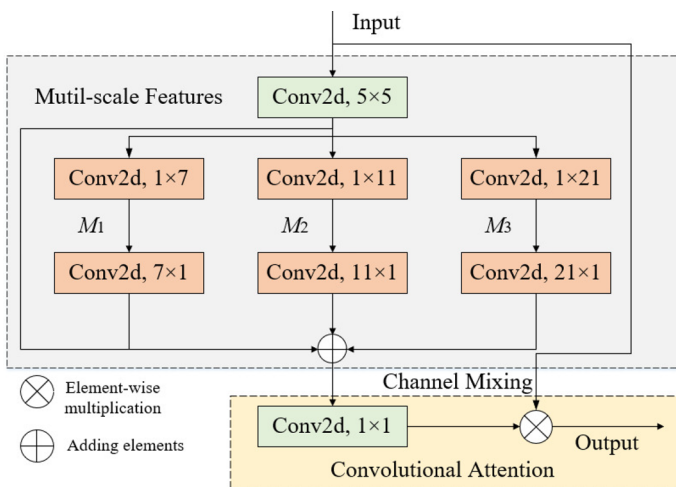


Fig. 15 MSCA structure

used as the weight to weight the input and perform the output. The MSCA structure is shown in Fig. 15.

The input feature F , enters the MSCA Module, as shown in formula (24), and the output feature is output, which is multiplied by the channel with the feature F , and the output feature is F'

$$F' = \text{Output} \otimes F \quad (25)$$

F' then enters the spatial attention module, the output feature $\text{SAM}(F)$ and feature F , carry out channel multiplication, and the output feature is F'' .

$$F'' = \text{SAM}(F) \otimes F' \quad (26)$$

where $\otimes$ represents element-wise multiplication and F'' is the final extracted output.

The MSAM structure is shown in Fig. 16.

In order to verify the effectiveness of MSAM attention mechanism, it was compared with other four mainstream attention mechanisms CBAM, CA, GAM and SimAM. The specific method is to embed these five attention mechanisms into the Backbone of YOLOv8n, and compare and analyze the model performance through a unified experimental setup. The experimental results can intuitively show the advantages of MSAM in model checking accuracy.

3.3 GSConv module

GSConv [26] is introduced as a replacement for convolution. Its optimized convolutional structure reduces the computational cost to 60%-70% of that of convolution, significantly improving inference efficiency while reducing redundant information processing. GSConv combines the outputs of standard and depthwise convolutions, and employs concat and shuffle operations to further enhance feature interaction and fusion capabilities, effectively alleviating the issue of information fragmentation caused by feature map concatenation.

GSConv is first input for downsampling of a normal convolution, then DWConv is used for depth convolution, and the results of the two convolutions are concatenated. Finally, the shuffle operation is performed, so that the corresponding channel numbers of the previous two convolutions are next to each other. The concat and

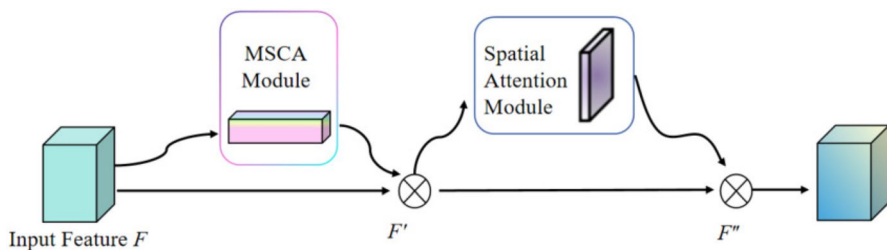


Fig. 16 MSAM structure

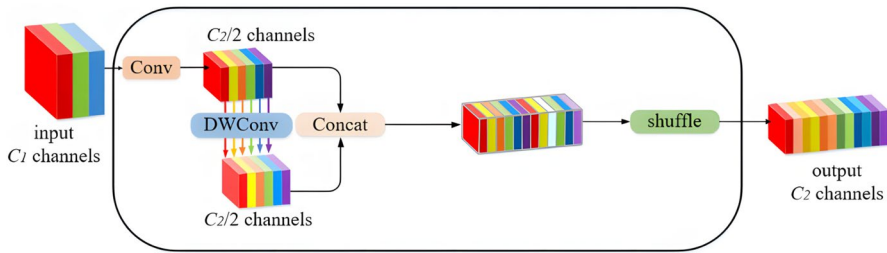


Fig. 17 GSConv structure

shuffle operations make the output of GSConv closer to that of standard convolution, enhancing the feature extraction and fusion capabilities of GSConv. Figure 17 shows the GSConv structure.

Input feature Y and channel number C_1 , perform Conv operation, and generate feature graph Y_{conv} with channel number $C_2/2$.

$$Y_{\text{conv}} = \text{Conv}(Y) \quad (27)$$

Then another feature graph Y_{dw} with the same channel number $C_2/2$ is obtained through DWConv module.

$$Y_{\text{dw}} = \text{DWConv}(Y_{\text{conv}}) \quad (28)$$

The output Y_{conv} of Conv and output Y_{dw} of DWConv are concatenated in the channel dimension.

$$Y_{\text{concat}} = \text{Concat}(Y_{\text{conv}}, Y_{\text{dw}}) \quad (29)$$

Finally, the feature map Y_{output} with C_2 output channels is obtained by Shuffle operation.

$$Y_{\text{output}} = \text{shuffle}(Y_{\text{concat}}) \quad (30)$$

In order to deeply analyze the effectiveness of the GSConv module in improving the performance of target detection, GSConv was embedded and tested against three other advanced convolutional techniques - DCNv3, SAConv, and SCConv. Through this comparison, the aim is to demonstrate the superiority of GSConv over other methods when it comes to target detection tasks.

3.4 SPPFPool module

The main purpose of using SPPF module is to enhance feature extraction by adopting multiple pooling kernel sizes, so that the network can capture more diverse feature information, thereby improving the recognition accuracy of the model. However, SPPF mainly focuses on edge information in the image—that is, local features such as contour, texture, and structural changes of objects. Edge information is very

important for accurate location and segmentation of objects, but in some cases, relying solely on edge information may not fully capture the background information and environmental relationships of objects, resulting in misjudgment of the model in complex scenes.

In the task of railway fastener detection, "background information" refers to the railway track environment where the fasteners are located and the spatial relationships between the fasteners and surrounding facilities. For example, sleepers and rails belong to the background information, which provide references for the installation position, arrangement pattern of the fasteners, and the overall structure of the railway track. During the detection process, this background information can help the model understand whether the size, position, and shape of the fasteners are in line with the normal operating state of the railway track. "Context information" further refers to the connection between this background information and the fastener targets, especially the relative positions of the fasteners to the sleepers or rails, and the regularity of the fasteners in the overall layout of the railway track. For instance, when there are interfering objects such as ballast on the railway track, the edge features of these interfering objects may be similar to the shape of the fasteners, which can easily lead to false detections. However, through context information, the model can combine the spatial relationships between the fasteners, sleepers, and rails, as well as the global pattern of fastener arrangements, to accurately distinguish interfering objects from actual fasteners, thus reducing misjudgments.

In order to make up for the limitation that SPPF ignores background and context information when extracting edge information, this paper proposes an extended version of SPPFPool based on SPPF module. SPPFPool effectively introduces global background information and context information into the feature extraction process by combining the global average pooling (GAP) and global max pooling (GMP) layers. The global average pooling captures the overall background and context information of the image and provides a global scene understanding, while the global max pooling focuses on retaining the most significant features in the image to ensure the accurate extraction of edge information. Through these two pooling methods, SPPFPool not only enhances the attention to the edge of the object, but also integrates the global background and context information, so that the network can better understand the position and relationship of the objects in the overall scene, so as to make more accurate judgments in complex scenes.

This comprehensive utilization of edge, background and context information makes SPPFPool more adaptable in object detection tasks, especially in the face of objects with different scales and different backgrounds, it can effectively reduce false detections and missed detections, and improve detection accuracy. The SPPFPool structure is shown in Fig. 18, which combines spatial pyramid pooling and global pooling strategies to provide a more balanced and context-aware feature fusion process.

The input feature map is X , and the convolution operation is performed.

$$Y_{\text{conv}} = \text{Conv}(X) \quad (31)$$

Pooling operations for different paths.

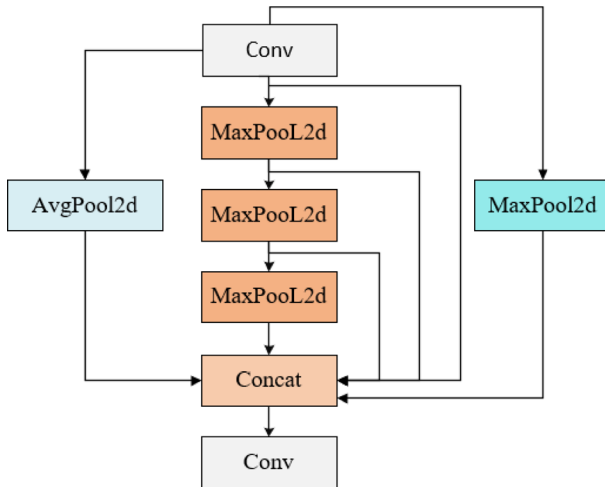


Fig. 18 SPPFPool structure

Average pooling path

$$Y_{\text{avgpool}} = \text{AvgPool2d}(Y_{\text{conv}}) \quad (32)$$

Max pooling paths

$$Y_{\text{maxpool1}} = \text{MaxPool2d}(Y_{\text{conv}}) \quad (33)$$

$$Y_{\text{maxpool2}} = \text{MaxPool2d}(Y_{\text{maxpool1}}) \quad (34)$$

$$Y_{\text{maxpool3}} = \text{MaxPool2d}(Y_{\text{maxpool2}}) \quad (35)$$

Max pooling on the far right

$$Y_{\text{maxpool-right}} = \text{MaxPool2d}(X) \quad (36)$$

Concatenate all pooling and convolution results.

$$Y_{\text{concat}} = \text{Concat}(Y_{\text{avgpool}}, Y_{\text{conv}}, Y_{\text{maxpool1}}, Y_{\text{maxpool2}}, Y_{\text{maxpool3}}, Y_{\text{maxpool-right}}) \quad (37)$$

Final convolution operation.

$$Y_{\text{output}} = \text{Conv}(Y_{\text{concat}}) \quad (38)$$

SPPFPool was compared with SimSPPF, ASPP, SPPCSPC and SPPELAN to verify the advantages of SPPFPool module in improving the performance of the model.

3.5 FocalerIoU loss function

The FocalerIoU [27] loss function is introduced as a replacement for the YOLOv8n CIoU loss function, addressing critical challenges in bounding box regression for target detection tasks. FocalerIoU incorporates a focal loss mechanism that dynamically adjusts the contribution of hard-to-classify samples during training. By introducing a focal parameter, it reduces the overwhelming influence of easily predictable samples while amplifying the focus on challenging ones. This targeted weighting strategy ensures that the model learns effectively from difficult cases, addressing the issue of imbalanced sample distribution, which often hinders traditional regression methods.

FocalerIoU enhances convergence speed and stability by promoting smoother gradients and improved optimization dynamics. It effectively balances localization and scale sensitivity, making it particularly well-suited for scenarios involving objects of varying sizes and aspect ratios. Its superior adaptability to complex environments further underscores its advantages, enabling the model to achieve higher detection accuracy while mitigating performance degradation caused by noisy or ambiguous data.

The linear interval mapping method reconstructs the IoU loss as shown in (39).

$$\text{IoU}^{\text{focaler}} = \begin{cases} 0, & \text{IoU} < d \\ \frac{\text{IoU} - d}{u - d}, & d \leq \text{IoU} \leq u \\ 1, & \text{IoU} > u \end{cases} \quad (39)$$

When IoU is less than a lower threshold d , the loss is 0; when IoU is greater than an upper threshold u , the loss is 1; and when IoU is between d and u , the loss is a function that increases linearly according to the value of IoU.

The FocalerIoU formula is shown in (40).

$$L_{\text{FocalerIoU}} = 1 - \text{IoU}^{\text{focaler}} \quad (40)$$

By comparing the performance indicators of three loss functions: EIou, SIou and FocalerIoU, the advantages of FocalerIoU in optimizing object detection tasks are verified.

4 Experiment

4.1 Experimental environment

The model runs on the Linux Ubuntu operating system, equipped with a NumXeon(R) Platinum 8255C CPU and an RTX 4090 GPU with 24GB of video memory. The deep learning framework used is PyTorch 1.11.0, with Python version 3.8 and CUDA version 12.0. The training parameters are configured as follows: a learning rate of 0.01, a batch size of 16, 300 training epochs, an IoU threshold of 0.5, and an input image size

Table 1 Experimental environment

Name	Related configurations
Operating system	Linux Ubuntu
CPU	NumXeon(R) Platinum 8255C
GPU	RTX4090
Video memory	24 GB
Framework	Pytorch-1.11.0
Python	Version 3.8
CUDA	Version 12.0
Learning rate	0.01
Batchsize	16
Epoch	300
Iou threshold	0.5
Image size	640×640
Optimizer	SGD
Confidence threshold	0.25
Momentum	0.937

of 640×640. The optimizer selected is SGD, with a confidence threshold of 0.25 and a momentum parameter of 0.937.

The experimental running environment of this paper is shown in Table 1.

4.2 Evaluation indicators

This study uses evaluation criteria for deep learning including: precision, recall, mAP, and FPS, FLOPs, parameters, training time.

Precision: the ratio of the number of samples correctly predicted as positive to the total number of samples predicted as positive. High precision means accurate prediction. The formula is shown in (41).

$$P = \frac{TP}{TP + FP} \quad (41)$$

Recall: the ratio of correct samples correctly identified by the model to the number of all correct samples actually present. The higher the recall, the less the model misses in detecting the target. The formula is shown in (42).

$$R = \frac{TP}{TP + FN} \quad (42)$$

mAP: multiple categories of APs were averaged to obtain a high mAP value, indicating good accuracy of the model. Among them, mAP@0.5 represents the average mAP at an IoU threshold of 0.5, mAP@0.5:0.95 represents the average mAP at different IoU thresholds (0.5–0.95, step size 0.05). The formula is shown in (44).

$$AP = \int_0^1 P(R) dR \quad (43)$$

$$mAP = \frac{1}{n} \sum_{i=0}^n AP_i \quad (44)$$

where *TP*(True Positive) denotes the number of train track fasteners and defects correctly detected, *FP*(False Positive) denotes the number of train track fasteners and defects misdetected, and *FN*(False Negative) denotes the number of train track fasteners and defects missed.

FPS denotes the number of image frames per second that the model can process, which directly reflects the real-time performance and efficiency of the target detection algorithm, a high FPS target detection algorithm means that it is able to process the image faster and identify and locate the target faster.

FLOPs represents the computational complexity of the model and represents the number of floating-point operations involved in executing the model in G. In general, the smaller the FLOPs, the lower the computational overhead of the model.

Parameters represent the number of parameters of a model. Models with a large number of parameters often require more computing resources, which slows down the inference speed of the model and increases the training time. In the text, param is used to represent "parameters", and the unit is Million (M).

Training time refers to the total time spent from the start of model training until the model converges or reaches the predetermined number of training epochs.

4.3 Dataset

In this paper, the train track fastener defective data images for 2080, contains three target categories: normal, broken, move, which normal means that the fastener bar is completely undamaged and the position is correct, broken means that the fastener bar is damaged; move means that the fastener bar position is shifted.

The training set, validation set, and test set are randomly partitioned at a ratio of 8:1:1. The dataset is annotated with MakeSense software to generate txt label files suitable for YOLOv8n training. The fasteners are of type m. The image categories from left to right are normal, broken, and move, respectively. Figure 19 presents the number of labels and some images of this dataset.

4.4 Experimental results and analysis

4.4.1 Comparative analysis of multiple attention results

The application effects of five different types of attention mechanisms in YOLOv8n are comprehensively analyzed and compared, and the results are shown in Table 2.

MSAM has a significant advantage on mAP under different IoU criteria. When the IoU is 0.5, the mAP is 79.1%, which indicates that the target can be located

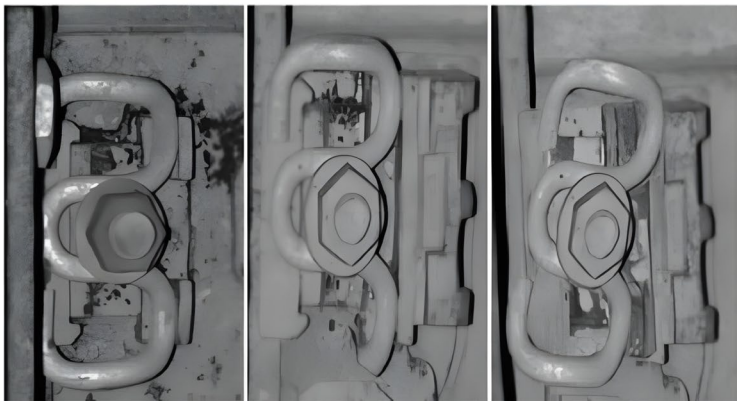
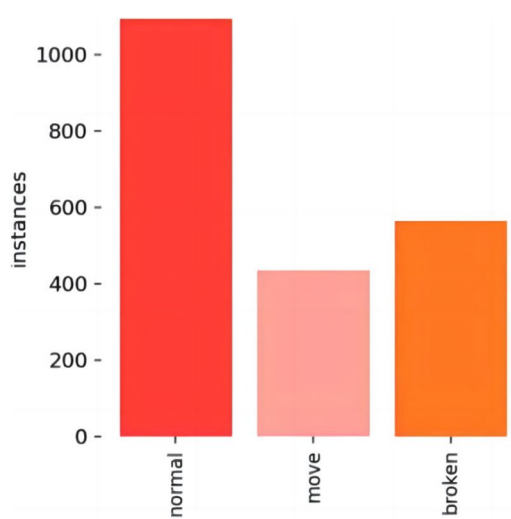


Fig. 19 Number of labels and fastener images

Table 2 Comparison of the results of attention mechanisms

Model	P/%	R/%	mAP		FPS	Param/M	FLOPs/G	Training time/h
			@0.5/%	@0.5:0.95/%				
YOLOv8n	73.4	76.2	77.5	41.3	102	3.20	8.3	1.492
+CBAM	72.1	77.1	76.3	39.8	97	3.31	8.4	1.543
+GAM	70.6	76.8	74.6	40.2	90	3.57	8.7	1.630
+SimAM	69.2	77.1	72.5	38.2	100	4.46	9.2	1.758
+CA	73.2	75.3	73.8	39.5	93	3.25	8.3	1.801
+MSAM	74.2	77.8	79.1	42.8	98	3.34	8.5	1.682

and identified more accurately. The gap in this metric compared to other attention reflects the ability of MSAM to accurately detect objects. Under the stricter IoU standard (0.5–0.95), the mAP of MSAM is 42.8%, which is ahead of other models. This shows that MSAM can detect the target more accurately in complex scenes and different target scales, and reduce the cases of false detection and missed detection.

A recall rate of 77.8% indicates that MSAM can better capture the real targets in the images and detect more real targets compared to other attention mechanisms.

The FPS of MSAM reaches 98, which is at a relatively high level among the models with added attention mechanisms. This enables MSAM to process image frames rapidly while ensuring detection accuracy, achieving fast response and real-time feedback. The performance with 3.34 M parameters and 8.5G computational complexity indicates that MSAM has properly controlled the model complexity. Compared with other models, it neither significantly increases the number of parameters and computational complexity in the pursuit of high accuracy nor sacrifices too much accuracy to reduce the computational cost. It strikes a good balance between the deployability and performance of the model, facilitating deployment and application on different hardware devices. The training time of 1.682 h means that training can be completed more quickly during the model development and iteration process, facilitating rapid experimentation and optimization. The training curve in Fig. 20 reflects the advantages of adding the MSAM attention mechanism.

In conclusion, the comprehensive performance of the MSAM attention mechanism in terms of accuracy, speed, complexity and training cost is better than that of other attention mechanisms.

The MSAM module significantly improves the performance of the model by dynamically allocating attention weights. To more fully evaluate the performance of the module under noisy conditions, experiments are designed and presented in Fig. 21, which presents the performance of the test model under noisy image input. Analysis results show that the dynamic weight allocation of MSAM module is sensitive to noisy image input, and this sensitivity is helpful to improve the robustness and detection accuracy of the model.

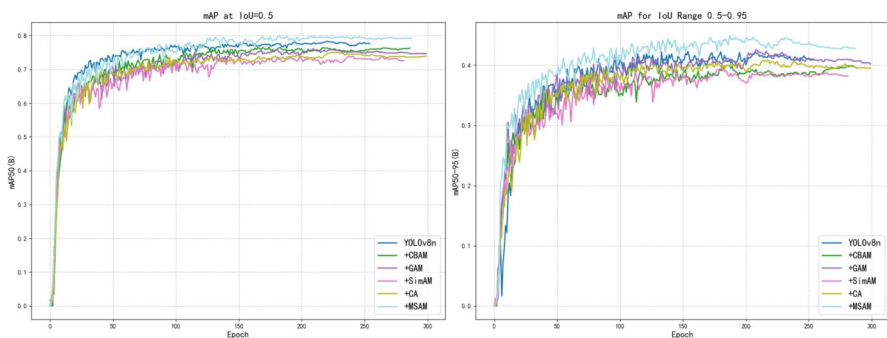


Fig. 20 Comparison of training curves of different attention mechanisms

Fig. 21 Noisy image test comparison

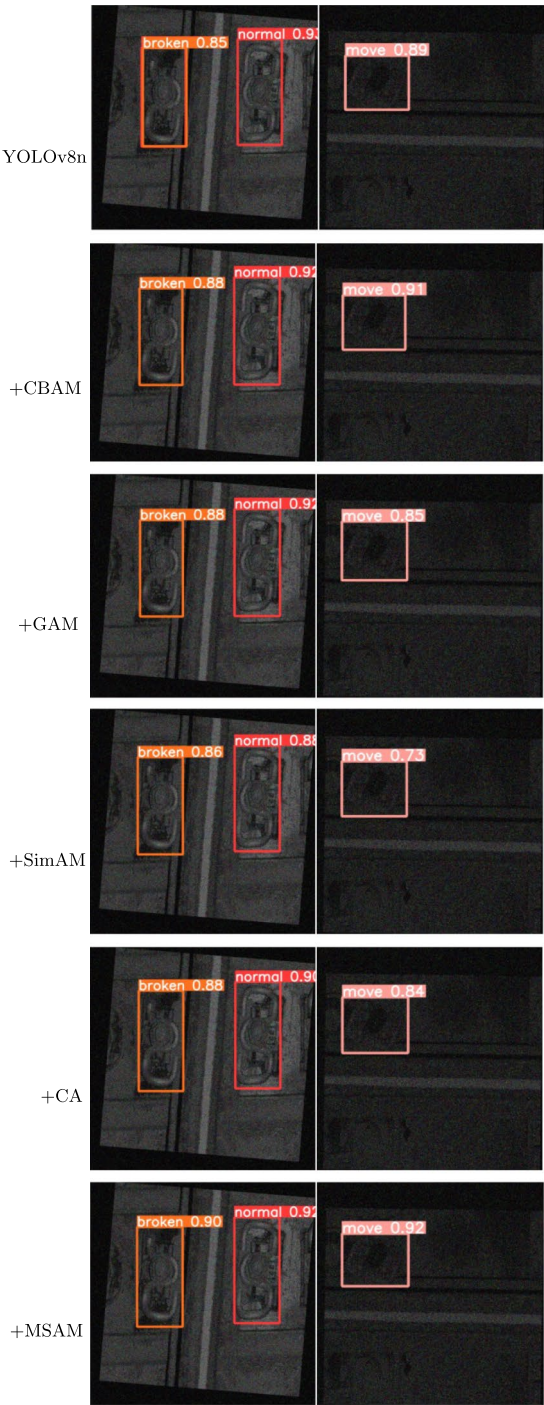


Table 3 Comparing the results of different convolution modules

Model	P/%	R/%	mAP		FPS	Param/M	FLOPs/G	Training time/h
			@0.5/%	@0.5:0.95/%				
YOLOv8n	73.4	76.2	77.5	41.3	102	3.20	8.3	1.492
+SAConv	72.8	73.2	70.4	38.7	88	4.72	7.3	1.745
+SCConv	68.0	73.1	68.6	37.1	92	2.64	7.7	1.702
+DCNv3	72.2	77.3	76.6	41.0	103	3.34	8.4	1.521
+GSConv	73.0	78.3	78.6	41.6	109	2.72	7.6	1.537

4.4.2 Comparison of multiple convolutional results

The comparison results of introducing four different convolutions are shown in Table 3. In addition to having a slightly lower p-value than YOLOv8n, GSConv has different degrees of advantages over other methods in R and mAP indicators, especially leading SAConv and SCConv on mAP@0.5, which proves its excellent detection accuracy. The FPS of GSConv is 109, which is 7 FPS higher than that of YOLOv8n and 21 FPS higher than that of SAConv. 17 FPS higher than SCConv 6 FPS higher than DCNv3. It shows that it has obvious advantages in detection speed and can process images faster to meet the needs of real-time applications. The overall calculation amount of GSConv is in a reasonable range, and it has advantages compared with some models, indicating that GSConv is more efficient in the utilization of computing resources. Although the training time is slightly longer than that of YOLOv8n and DCNv3, it is shorter than that of SAConv and SCConv and has certain advantages in training efficiency. The training curve in Fig. 22 illustrates the advantage of GSConv in mAP.

In conclusion, GSConv shows good comprehensive performance in comparison with other models.

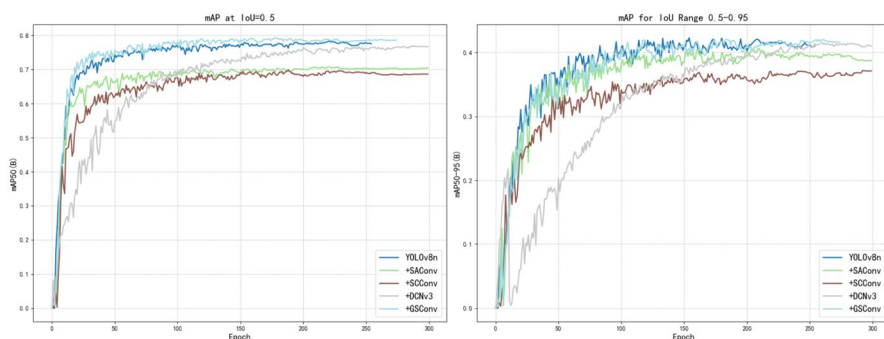
**Fig. 22** Comparison of different convolutional training curves

Table 4 Compare the results of different spatial pyramid pooling modules

Model	P/%	R/%	mAP		FPS	Param/M	FLOPs/G	Training time/h
			@0.5/%	@0.5:0.95/%				
YOLOv8n	73.4	76.2	77.5	41.3	102	3.20	8.3	1.492
+ASPP	67.6	70.1	67.8	35.2	87	3.83	8.9	1.596
+SPPCSPC	68.8	77.9	70.6	39.0	90	4.61	9.6	1.341
+SimSPPF	72.3	74.1	73.3	38.6	94	3.32	8.4	1.435
+SPPELAN	71.5	78.7	76.3	39.8	107	3.37	8.4	1.422
+SPPFPool	73.6	79.1	78.7	42.1	100	3.27	8.3	1.831

4.4.3 Comparative analysis of multiple spatial pyramid pooling

The comparison results of five different spatial pyramid pooling are shown in Table 4. SPPFPool has different degrees of improvement compared with other methods. Especially, it leads ASPP and SPPCSPC by a large margin in mAP@0.5, indicating its outstanding performance in the accuracy of object detection. The FPS of SPPFPool is 100, which is slightly lower than that of YOLOv8n and SPPELAN. However, compared with ASPP, SPPCSPC and SimSPPF, it has a significant advantage in detection speed, can process images relatively quickly, and meet certain real-time requirements.

The number of parameters of SPPFPool is 3.27M. Except for being slightly higher than that of YOLOv8n, it is lower than that of other comparison models, which means that the model structure is relatively simple and it has more advantages in scenarios with limited storage resources. The computational complexity is within a reasonable range and has an advantage compared with some models, indicating that it is more efficient in utilizing computational resources. The training time of SPPFPool is 1.831 h. In terms of training time, SPPFPool is longer than other models, which is its relative disadvantage. The results of the training curves in Fig. 23 confirm the advantages possessed by SPPFPool in terms of mAP@0.5 and mAP@0.5:0.95.

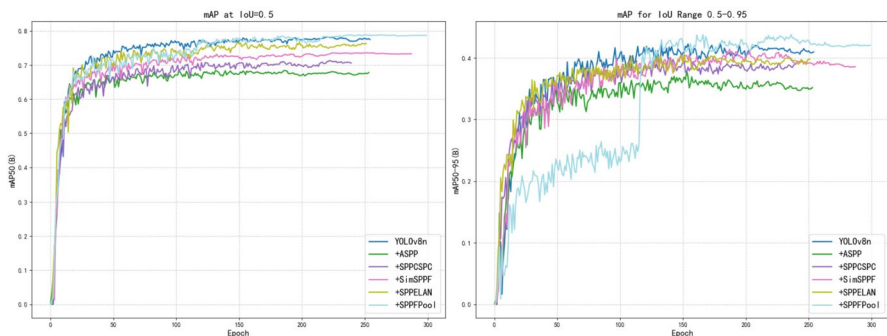
**Fig. 23** Comparison of different spatial pyramid pooling training curves

Table 5 Multiple loss function results comparison

Model	P/%	R/%	mAP		FPS	Param/M	FLOPs/G	Training time/h
			@0.5/%	@0.5:0.95/%				
YOLOv8n	73.4	76.2	77.5	41.3	102	3.20	8.3	1.492
+SIoU	65.9	75.4	70.8	36.8	96	3.20	8.3	1.290
+EIoU	71.0	74.5	72.5	38.7	92	3.20	8.3	1.472
+FocalerIoU	74.7	76.9	78.4	40.8	107	3.20	8.3	1.042

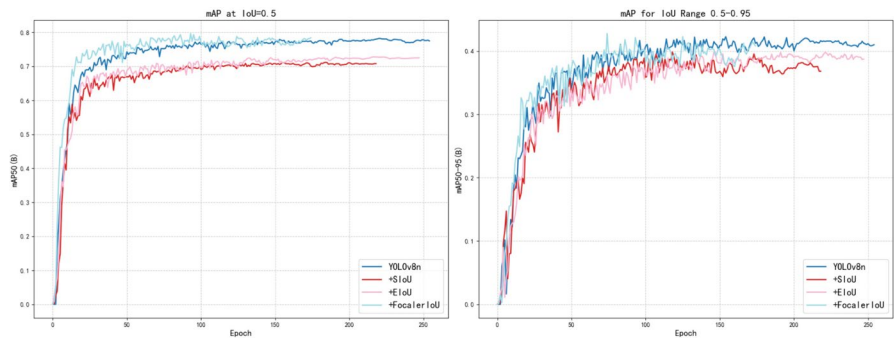


Fig. 24 Comparison of training curves for different loss functions

On the whole, although SPPFPool has a relatively long training time, its overall comprehensive performance is relatively excellent among these comparison models.

4.4.4 Comparative analysis of results of multiple loss functions

It can be seen from Table 5 that the introduction of FocalerIoU has a certain improvement in Precision, Recall and mAP@0.5 compared with YOLOv8n and other loss functions, and the gap with YOLOv8n is relatively small in mAP@0.5:0.95. The FPS of FocalerIoU is 107, which is 5 FPS higher than that of YOLOv8n, showing an advantage in speed. There is no difference in the number of parameters and computational complexity among various methods. The training time of FocalerIoU is 1.042 h, which is 0.45 h less than that of YOLOv8n. FocalerIoU has the shortest training time and has a significant advantage in training efficiency. The training curve results in Fig. 24 strongly confirm the significant advantages of FocalerIoU in mAP@0.5 and mAP@0.5:0.95. In summary, FocalerIoU has better comprehensive performance in these comparison methods.

Therefore, MSAM attention mechanism, GSConv module, SPPFPool, FocalerIoU loss function are selected as the improvement points of this paper.

4.5 Ablation experiment

MSAM attention mechanism is added to the backbone network Backbone, GSConv convolution is used to replace Conv, SPPFPool is introduced, and FocalerIoU is introduced. In order to verify the positive effect of these improvements on the model performance, ablation experiments are conducted to add these improvements and train the model step by step. These experiments are designed to demonstrate the positive effect of adding the improvement points. The specific results of the ablation experiments are shown in Table 6.

Adding MSAM attention mechanism, replacing Conv with GSConv convolution, introducing SPPFPool, FocalerIoU, to improve the performance of the model, the mAP is improved by 1.6%, 1.1%, 1.2%, 0.9%, respectively, compared to YOLOv8n. The experimental results show that all these improvement modules have positive impact on the network to enhance the detection precision.

After integrating MSAM, GSConv, SPPFPool, and FocalerIoU, the model's performance metrics show significant improvements compared to YOLOv8n. Precision increased from 73.4% to 78.1%, a gain of 4.7 percentage points; recall improved from 76.2% to 80.5%, up by 4.3 percentage points; mAP@0.5 rose from 77.5% to 80.1%, an increase of 2.6 percentage points; and mAP@0.5:0.95 went up from 41.3% to 43.8%, a boost of 2.5 percentage points. Additionally, FPS increased by 8. Incorporating these modules has significantly enhanced the model's precision, recall, and mAP, leading to superior overall performance. The YOLOv8n, MGSF-YOLO train track fastener inference results are shown in Fig. 25.

The training loss graph of YOLOv8n, MGSF-YOLO-without FocalerIoU, and MGSF-YOLO-with FocalerIoU models is compared. It can be seen from Fig. 26 that the convergence speed of the MGSF-YOLO model is significantly faster than that of the MGSF-YOLO model and YOLOv8n without FocalerIoU after adopting FocalerIoU. BoxLoss, which measures the gap between the actual bounding box and the predicted bounding box of the target object, and ClassificationLoss, which measures the accuracy of the model's prediction for each target object class, are also significantly lower. The DFL Loss (Distribution Focal Loss), which is used to correct

Table 6 Results of ablation experiments

Model	MSAM	GSConv	SPPFPool	FocalerIoU	P/%	R/%	mAP/%		FPS
							@0.5	@0.5:0.95	
YOLOv8n	×	×	×	×	73.4	76.2	77.5	41.3	102
2	✓	×	×	×	74.2	77.8	79.1	42.8	98
3	×	✓	×	×	73.0	78.3	78.6	41.6	109
4	×	×	✓	×	73.6	79.1	78.7	42.1	100
5	×	×	×	✓	74.7	76.9	78.4	40.8	107
6	✓	✓	×	×	74.6	76.4	79.5	43.2	111
7	✓	✓	✓	×	77.8	80.2	79.7	43.5	106
MGSF-YOLO	✓	✓	✓	✓	78.1	80.5	80.1	43.8	110

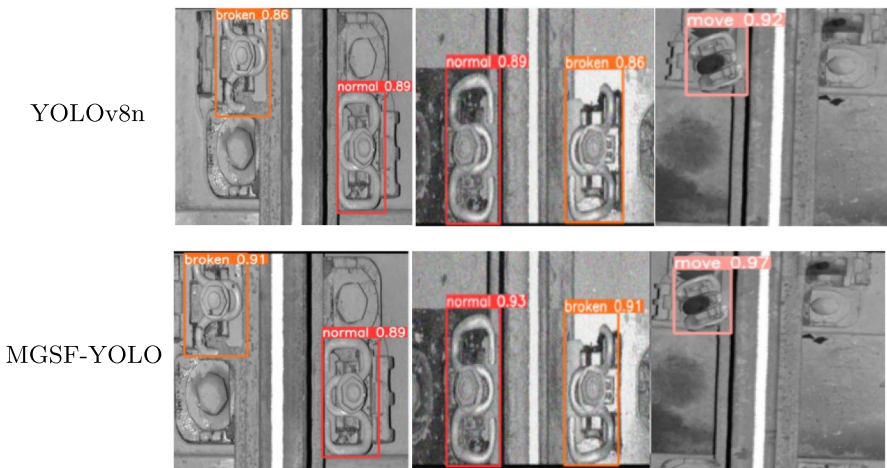


Fig. 25 Track fastener image test comparison

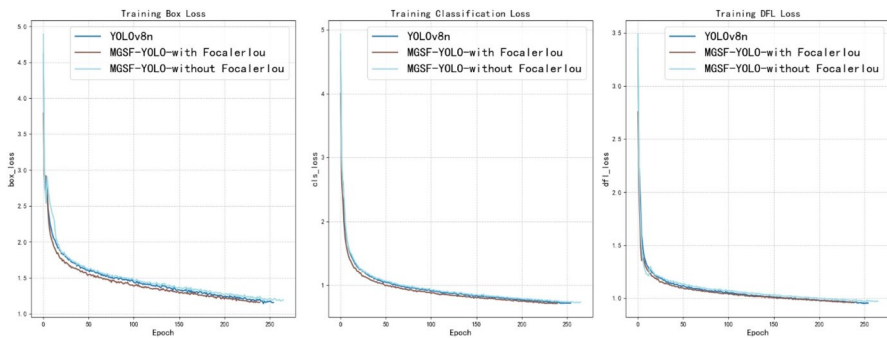


Fig. 26 Model training loss plot

the error of the model in predicting the object bounding box, is also similar to that before modification in training, indicating that the MGSF-YOLO model is better than the original model YOLOv8n on the track fastener image dataset.

4.6 Comparative experiment

The detection results of the two models YOLOv8n, MGSF-YOLO for different categories normal, move and broken are shown in Table 7.

Compared to YOLOv8n, MGSF-YOLO exhibits a 1.5% higher precision, a 9.1% higher recall, and a 2.7% higher AP in normal category detection. In the detection of move and broken categories, MGSF-YOLO surpasses YOLOv8n with a 6.7% and 5.9% higher precision, and a 0.6% and 4.5% higher AP, respectively. These data indicate that MGSF-YOLO consistently outperforms YOLOv8n in overall detection performance.

Table 7 Comparison of results by category

Categories	P/%	R/%	AP/%
<i>YOLOv8n</i>			
normal	77.8	72.5	77.1
move	70.5	77.2	76.3
broken	71.9	78.9	79.1
<i>MGSF-YOLO</i>			
normal	79.3	81.6	79.8
move	77.2	76.7	76.9
broken	77.8	83.2	83.6

To further verify the advantages of MGSF-YOLO, sample images of the following four typical cases are listed in the figure. The left image is the YOLOv8n detection result, and the right image is the MGSF-YOLO detection result.

(1) TP samples

Figure 27 shows that both MGSF-YOLO and YOLOv8n can correctly detect the target sample, but MGSF-YOLO has higher confidence.

(2) FP samples

Figure 28 indicates that YOLOv8n has misjudgment, predicting fasteners damage as normal fasteners, but MGSF-YOLO correctly detects the target sample.

(3) FN samples

Figure 29 indicates that YOLOv8n does not detect the target fastener sample, while MGSF-YOLO can successfully detect the fastener sample.

**Fig. 27** TP samples

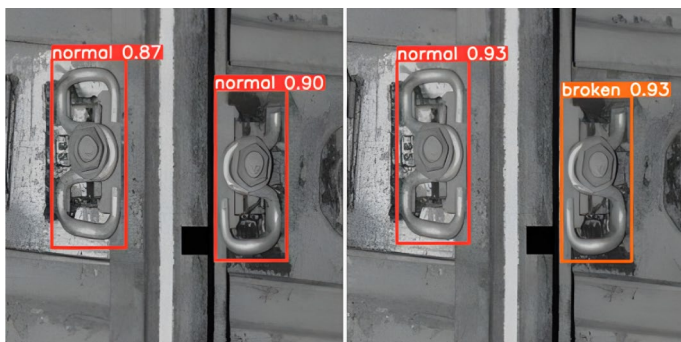


Fig. 28 FP samples

Fig. 29 FN samples

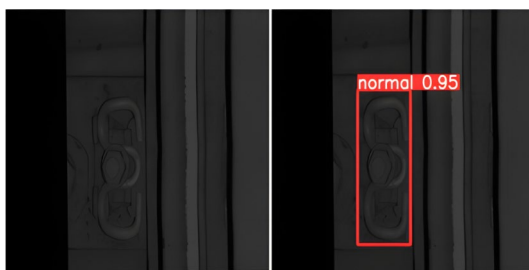
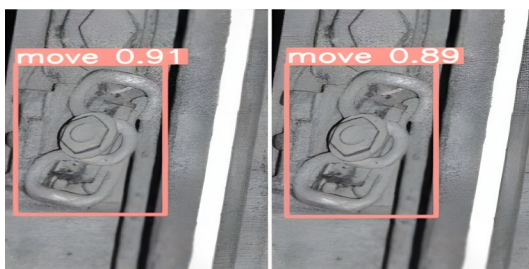


Fig. 30 TN samples



(4) TN(True Negative) samples

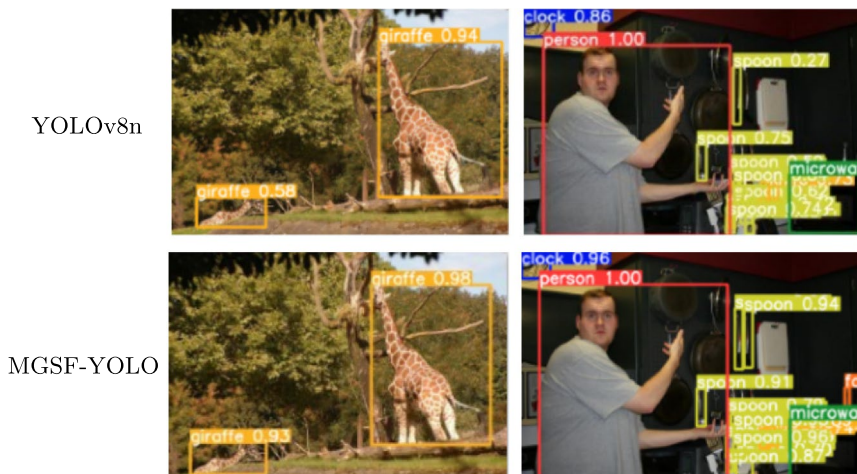
Figure 30 indicates that both MGSF-YOLO and YOLOv8n are able to accurately identify the fastener area.

Through the detailed comparison of the above samples, it can not only intuitively show the performance differences between the two models in various detection tasks, but also further verify the effect of MGSF-YOLO in improving detection accuracy and reducing false positive and missed detection.

To comprehensively evaluate the detection performance of the improved MGSF-YOLO algorithm, validation was conducted using the public datasets coco128 and Northeastern University's NEU-DET open dataset. The coco128

Table 8 Comparison of results in coco128 dataset

Model	P/%	R/%	mAP@0.5/%	mAP@0.5:0.95/%	FPS
YOLOv8n	93.3	84.4	89.7	71.7	65
MGSF-YOLO	93.8	85.7	91.3	73.4	73

**Fig. 31** Test comparison of coco128 datasets

dataset, a subset selected by Microsoft from the coco dataset, is a small-scale dataset consisting of the first 128 images from coco train2017. The NEU-DET dataset contains six types of surface defects on hot-rolled steel strips: crazing (Cr), inclusion (In), patches (Pa), pitted_surface (Ps), rolled-in_scale (Rs), and scratches (Sc), with 300 images per defect type, totaling 1800 images.

After 1000 rounds of iteration on the coco128 dataset, the detection results are shown in Table 8. The mAP of MGSF-YOLO reached 91.3%, which is 1.6% higher than that of YOLOv8n. The test results of YOLOv8n and MGSF-YOLO are shown in Fig. 31.

After 300 iterations, the model parameters are continuously optimized in each round to better fit the data characteristics. Finally, the detection results of the four models are summarized and clearly presented in Table 9. Table 9 covers the detection accuracy of each category and key evaluation indicators such as precision, recall, and mAP, which accurately measures the performance of each model in detecting different types of defects. Through data comparison, MGSF-YOLO has advantages in both single category detection accuracy and mAP. Its FPS is 95. Although this is lower than 108 of YOLOv11, it is higher than 80 of YOLOv8n and 92 of EC-YOLO. This indicates that while maintaining a high detection accuracy, its inference speed is also competitive and can meet certain real time requirements in practical applications. Two models, YOLOv8n and MGSF-YOLO, visualize their test results in Fig. 32.

Table 9 Comparison of results on NEU-DET dataset

Model	AP/%				P/%	R/%	mAP@0.5/%	mAP@0.5:0.95/%	FPS
	Cr	In	Pa	Ps					
YOLOv8n	49.3	93.5	93.7	90.4	78.5	96.8	76.0	83.7	52.3
YOLOv10	51.7	93.7	96.0	91.2	79.3	94.5	81.0	84.4	53.2
YOLOv11	52.7	92.2	93.8	91.9	81.7	96.5	79.8	84.8	53.9
EC-YOLO[28]	56.9	86.7	93.3	90.3	76.9	96.5	81.4	83.4	46.8
MGSF-YOLO	54.2	95.4	94.0	94.9	80.4	97.7	79.7	86.1	55.3

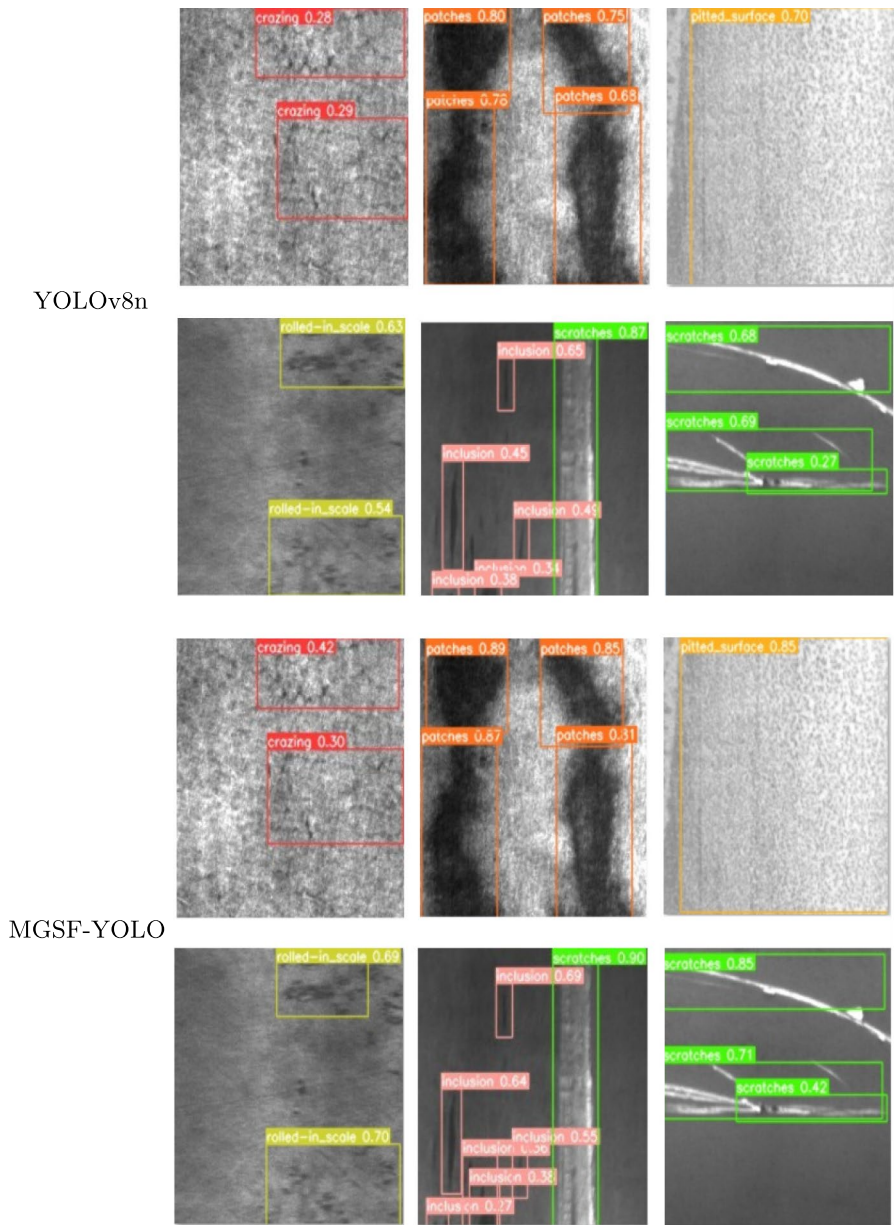
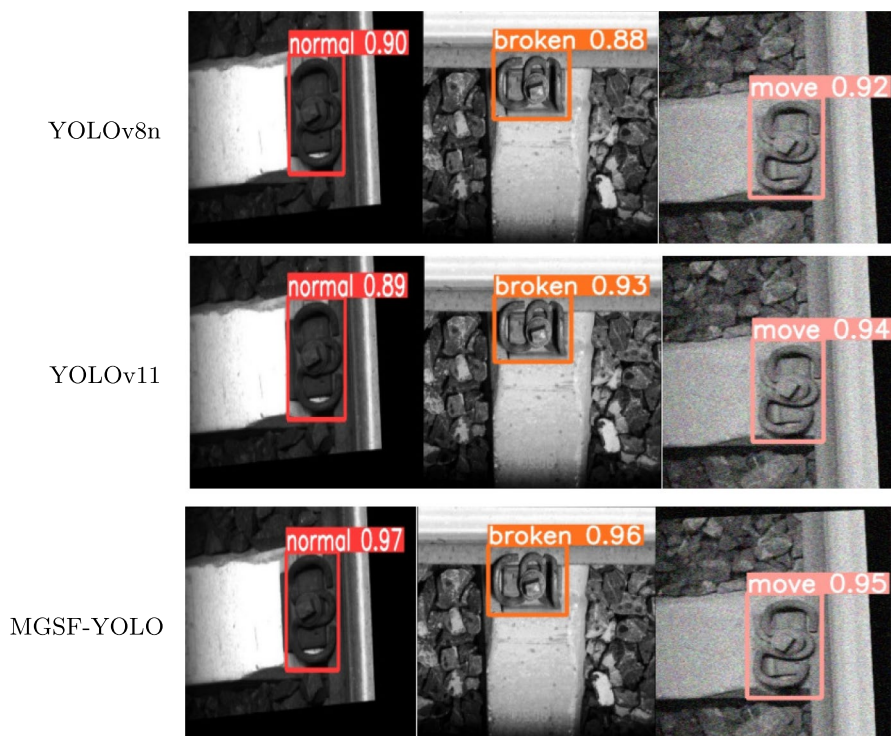


Fig. 32 NEU-DET datasets test comparison

MGSF-YOLO outperforms other models in multiple detection metrics and also has a good performance in terms of frame rate, demonstrating its excellent performance in object detection tasks.

Table 10 Comparison of results on fastener ground truth dataset

Model	P/%	R/%	mAP@0.5/%	mAP@0.5:0.95/%	FPS
YOLOv8n	90.2	88.6	90.7	67.3	130
YOLOv11	91.5	89.4	92.5	68.7	144
MGSF-YOLO	94.1	90.2	94.3	70.4	136

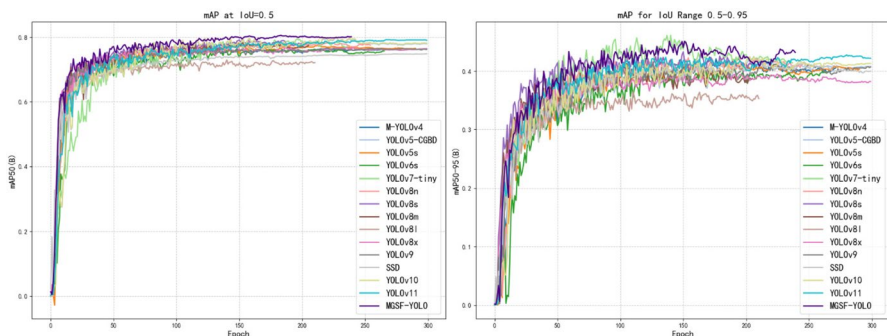
**Fig. 33** Fastener ground truth dataset test comparison

A ground truth dataset containing track fasteners was collected on the Roboflow website, with a total of 500 images, including 300 normal fasteners, 100 broken fasteners, and 100 shifted fasteners. This dataset was used for YOLOv8n, YOLOv11 and MGSF-YOLO models, respectively, and 100 rounds of iterative training were performed, and the detection results are shown in Table 10. The test results are shown in Fig. 33.

The mAP@0.5 of MGSF-YOLO is 94.3%, which is 3.6 percentage points higher than that of YOLOv8n and 1.8 percentage points higher than that of YOLOv11. In terms of mAP@0.5:0.95, MGSF-YOLO reaches 70.4%, 6.1 percentage points higher than YOLOv8n and 1.7 percentage points higher than YOLOv11. This reflects that MGSF-YOLO has better comprehensive detection performance under different IoU thresholds. Although the FPS of MGSF-YOLO is lower than 144 of YOLOv11, the

Table 11 Comparison with other target detection algorithms

Model	P/%	R/%	mAP		FPS	Param/M	FLOPs/G	Training time/h
			@0.5/%	@0.5:0.95/%				
M-YOLOv4 [29]	76.4	74.0	76.2	40.5	101	6.11	13.5	1.821
YOLOv5-CGBD [12]	76.7	73.1	77.9	40.8	104	6.75	15.4	1.762
YOLOv5s [30]	75.4	74.2	76.5	40.3	103	7.01	16.5	1.853
YOLOv6s [31]	75.3	74.7	75.7	39.9	100	17.19	42.3	1.656
YOLOv7-tiny [32]	75.1	76.2	77.3	41.5	107	6.20	5.8	1.204
YOLOv8n [11]	73.4	76.4	77.5	41.0	102	3.20	8.3	1.492
YOLOv8s [11]	77.8	74.1	78.4	41.3	98	11.13	28.4	1.614
YOLOv8m [11]	79.8	73.2	77.4	38.7	87	25.84	78.7	1.642
YOLOv8l [11]	79.9	70.2	72.1	35.3	78	43.61	164.8	2.238
YOLOv8x [11]	80.1	72.2	76.4	38.5	73	68.13	257.4	4.644
YOLOv9 [22]	74.7	75.3	76.2	40.7	94	20.10	76.8	2.243
SSD [33]	74.1	73.5	74.8	40.0	90	24.16	61.4	1.923
YOLOv10 [34]	78.8	72.2	78.1	41.3	113	2.30	7.2	2.337
YOLOv11 [35]	78.3	78.5	79.0	42.2	117	2.58	6.3	1.591
MGSF-YOLO	78.1	80.5	80.1	43.2	110	2.81	7.8	1.625

**Fig. 34** Comparison of training curves for different models

gap is relatively small. Moreover, while maintaining a relatively high frame rate, MGSF-YOLO has obvious advantages in detection indicators such as precision, recall, and mAP.

To verify the rationality and effectiveness of the proposed MGSF-YOLO algorithm, it is compared and analyzed with other object detection algorithms on the train track fastener dataset. The comparison results and the mAP training curves are shown in Table 11 and Fig. 34.

It can be observed that the proposed MGSF-YOLO algorithm has the highest mAP, which is 80.1%, and a detection speed of 110 FPS. Although this speed is lower than 113 of YOLOv10 and 117 of YOLOv11, the gaps are relatively small.

This means the algorithm can maintain a relatively high detection speed while ensuring the detection quality. The MGSF-YOLO algorithm has 2.81 million parameters and a computational complexity of 7.8G. Compared with YOLOv8n, the number of parameters is reduced by 12.2% and the computational complexity is decreased by 6.1%. Compared with other models, the number of parameters and computational complexity of MGSF-YOLO are also relatively small. Comprehensive analysis indicates that, compared with other mainstream algorithms, the MGSF-YOLO algorithm demonstrates superior comprehensive performance, providing certain reference for the defect detection of train track fasteners in practical applications.

According to Table 12, it can be analyzed that the AP of MGSF-YOLO in the normal category is 79.8%, which is at the highest level among all models. The AP of MGSF-YOLO in the move category is 76.9%, higher than 76.3% of YOLOv8n. However, YOLOv8m performs better in this category, reaching 79.2%, but the gap between MGSF-YOLO and it is not significant. The AP of MGSF-YOLO in the broken category reaches 83.6%, which is the highest among all models. Variance reflects the degree of data dispersion. The variance of MGSF-YOLO is 7.53, which is relatively large compared to 0.50 of YOLOv10 and 1.39 of YOLOv8n, indicating that the fluctuation of its detection accuracy in different categories is relatively large. Overall, MGSF-YOLO demonstrates excellent performance in terms of AP across three categories, indicating its strong overall performance.

Table 12 Comparison of defect category results of each model

Model	AP/%			mAP/%	S^2
	normal	move	broken		
M-YOLOv4	72.3	75.4	80.9	76.2	12.65
YOLOv5-CGBD	76.5	75.2	82.0	77.9	8.69
YOLOv5s	73.4	74.8	81.3	76.5	11.85
YOLOv6s	75.3	77.6	74.2	75.7	2.01
YOLOv7-tiny	71.6	78.1	82.2	77.3	19.05
YOLOv8n	77.1	76.3	79.1	77.5	1.39
YOLOv8s	78.2	75.3	81.7	78.4	6.85
YOLOv8m	73.7	79.2	79.6	77.4	7.25
YOLOv8l	68.9	74.7	72.7	72.1	5.79
YOLOv8x	74.2	76.3	78.7	76.4	3.38
YOLOv9	79.3	71.5	77.8	76.2	11.42
SSD	69.7	78.3	76.4	74.8	13.61
YOLOv10	77.6	79.1	77.6	78.1	0.50
YOLOv11	79.2	77.4	80.4	79.0	1.52
MGSF-YOLO	79.8	76.9	83.6	80.1	7.53

5 Conclusion

In order to solve the problem of low accuracy of train track fastener defect detection algorithm, an improved YOLOv8n algorithm named MGSF-YOLO is proposed.

- (1) The algorithm designs a new attention mechanism MSAM, which can dynamically adjust the weight of channel features and spatial locations, so that the model can focus more on key features and important regions, thereby significantly improving detection performance. In addition, the GSConv module is introduced to replace the Conv module, which not only reduces the number of parameters and computational overhead of the model, but also further improves the detection accuracy. At the same time, the SPPFPool module is designed, which integrates the average pooling and maximum pooling strategies to effectively aggregate the edge, background and context information of the image, enhance the robustness of the model to different sizes of input, significantly reduce false detections and missed detections caused by image scale changes, and improve the detection accuracy. Finally, the FocalerIoU loss function was introduced to further optimize the accuracy of regression results and significantly accelerate the convergence speed of the model.
- (2) The test results on the train track fastener defect dataset show that the MGSF-YOLO algorithm has achieved improvement in multiple indicators compared with YOLOv8n. The number of model parameters is reduced by 12.2%, the amount of computation is reduced by 6.1%, and the mAP is increased by 2.6%. In addition, the detection speed reaches 110 FPS, which is 8 FPS faster than YOLOv8n, showing higher efficiency and real-time performance. In further verification, MGSF-YOLO also performs well on multiple public datasets. On the coco128 dataset, the mAP is improved by 1.6%. On NEU-DET dataset, the mAP is increased by 2.4%. On a real fastener dataset collected from the Roboflow website, mAP improves by 3.6%. MGSF-YOLO achieves the highest level of mAP compared with other mainstream algorithms. This fully proves the generalization ability and detection performance of MGSF-YOLO on different scenes and datasets. Overall, MGSF-YOLO not only performs well in the rail fastener defect detection task, but also shows strong adaptability on diverse datasets, which provides reliable technical support for efficient and accurate rail fastener defect detection.
- (3) At present, the improved model in this paper is only applicable to the identification of M-type rail fastener defects. Therefore, the subsequent work will further expand the dataset of railway fasteners, build a dataset containing rich fastener types, and improve the generalization ability of the model. Specifically, more fastener image data of different types and different working conditions will be

collected to ensure the diversity, representativeness and coverage of the dataset, and the accuracy of data annotation will be strengthened to improve the quality of the dataset. Secondly, according to the different characteristics of various fastener types, the design of the model will pay more attention to the refinement of feature extraction, so that it can adapt to the rapid recognition of different types of fasteners and accurately distinguish various types of defects. At the same time, considering the requirements of detection efficiency and real time performance in practical application scenarios, the existing model will be lightweight improved under the premise of maintaining high accuracy, and an efficient network architecture will be used to optimize the algorithm process to improve the processing speed and ensure that the model can achieve real-time detection on embedded mobile devices. In addition, with the continuous enrichment of datasets and the optimization of the model, verification tests in multiple scenarios and environments will be carried out in the future to ensure the stability, adaptability and accuracy of the model under different working conditions, so as to provide reliable technical support for the intelligent detection and real-time monitoring of railway fastener defects.

Author Contributions All authors were involved in the conception and design of the algorithms, data collection and analysis was done by RHL, theoretical analysis and experiments were done by SWM and HNH, the first draft of the manuscript was written by SWM and HNH, and all authors have read and agreed to the published version of the manuscript.

Funding This study was supported by the Key Laboratory Fund Project of National Defense Science and Technology (2022-JCJQ-L8-015-020), the Key Project of Scientific Research Project of Liaoning Provincial Department of Education (LJKZ0475), and the Innovation Support Program for High-level Talents of Dalian City (2022RJ03).

Data Availability The data used to support the findings of this study is available from the corresponding author upon request.

Declarations

Conflict of interest The authors declare no competing interests.

Ethical approval and informed consent This article does not contain studies with human participants or animals. Statement of informed consent is not applicable since the manuscript does not contain any patient data.

References

1. Zhong H, Liu L, Wang J, Fu Q, Yi B (2022) A real-time railway fastener inspection method using the lightweight depth estimation network. *Measurement* 189:110613

2. Meng Y, Yang J, Liu S, Xu W, Chen G, Niu Z, Wang M, Deng T, Qin Y, Han M et al (2022) Nano-fiber based self-powered flexible vibration sensor for rail fasteners tightness safety detection. *Nano Energy* 102:107667
3. Hu Q, Fang F, Yuan R-J (2023) Fastener partial looseness identification of the ballastless track by a time-domain bayesian approach. *Adv Struct Eng* 26(10):1835–1846
4. Li S, Jin L, Jiang J, Wang H, Nan Q, Sun L (2022) Looseness identification of track fasteners based on ultra-weak FBG sensing technology and convolutional autoencoder network. *Sensors* 22(15):5653
5. Liu J, Huang Y, Zou Q, Tian M, Wang S, Zhao X, Dai P, Ren S (2019) Learning visual similarity for inspecting defective railway fasteners. *IEEE Sens J* 19(16):6844–6857
6. Qiang W, Bailin L, Yun H, Hong F (2018) An improved LBP feature for rail fastener identification. *J Southwest Jiaotong Univ* 53(5):893–899
7. Bai T, Yang J, Xu G, Yao D (2021) An optimized railway fastener detection method based on modified faster R-CNN. *Measurement* 182:109742
8. Ge X, Qin Y, Cao Z, Gao Y, Lian L, Bai J, Yu H (2023) A fine-grained method for detecting defects of track fasteners using rgb-d image. In: *International Conference on Electrical and Information Technologies for Rail Transportation*, Springer, pp 37–44
9. Aydin I, Sevi M, Salur MU, Akin E (2022) Defect classification of railway fasteners using image preprocessing and alightweight convolutional neural network. *Turk J Electr Eng Comput Sci* 30(3):891–907
10. Su S, Du S, Wei X, Lu X (2022) RFS-Net: railway track fastener segmentation network with shape guidance. *IEEE Trans Circuits Syst Video Technol* 33(3):1398–1412
11. Sohan M, Sai Ram T, Reddy R, Venkata C (2024) A review on yolov8 and its advancements. In: *International Conference on Data Intelligence and Cognitive Informatics*, Springer, pp 529–545
12. Wang L, Zang Q, Zhang K, Wu L (2024) A rail fastener defect detection algorithm based on improved yolov5. In: *Proceedings of the Institution of Mechanical Engineers, Part F: Journal of Rail and Rapid Transit*, 09544097241234380
13. Hou Q, Zhou D, Feng J (2021) Coordinate attention for efficient mobile network design. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp 13713–13722
14. Zhao Y, Ju Z, Sun T, Dong F, Li J, Yang R, Fu Q, Lian C, Shan P (2023) Tgc-yolov5: an enhanced yolov5 drone detection model based on transformer, gam & ca attention mechanism. *Drones* 7(7):446
15. Yang L, Zhang R-Y, Li L, Xie X (2021) Simam: a simple, parameter-free attention module for convolutional neural networks. In: *International Conference on Machine Learning*, PMLR, pp 11863–11874
16. Wang W, Dai J, Chen Z, Huang Z, Li Z, Zhu X, Hu X, Lu T, Lu L, Li H, et al (2023) Internimage: exploring large-scale vision foundation models with deformable convolutions. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp 14408–14419
17. Tang J, Lei J, Xu D, Ma F, Jia K, Zhang L (2021) Sa-convnet: sign-agnostic optimization of convolutional occupancy networks. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp 6504–6513
18. Li J, Wen Y, He L (2023) Sconv: spatial and channel reconstruction convolution for feature redundancy. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp 6153–6162
19. Gupta C, Gill NS, Gulia P, Chatterjee JM (2023) A novel finetuned yolov6 transfer learning model for real-time object detection. *J Real-Time Image Proc* 20(3):42
20. Wang K, Wei Z (2022) Yolo v4 with hybrid dilated convolution attention module for object detection in the aerial dataset. *Int J Remote Sens* 43(4):1323–1344
21. Priya DK, Ramkumar M, Menaka D (2023) Rail fastener fault detection based on enhanced yolov7 model. In: *2023 International Conference on Circuit Power and Computing Technologies (ICCPCT)*, IEEE, pp 1681–1689
22. Wang C-Y, Yeh I-H, Mark Liao H-Y (2025) Yolov9: Learning what you want to learn using programmable gradient information. In: *European Conference on Computer Vision*, Springer, pp 1–21
23. Zhang Y-F, Ren W, Zhang Z, Jia Z, Wang L, Tan T (2022) Focal and efficient IOU loss for accurate bounding box regression. *Neurocomputing* 506:146–157
24. Zou H, Wang Z (2024) An enhanced object detection network for ship target detection in SAR images. *J Supercomput* 80:17377–17399

25. Guo M-H, Lu C-Z, Hou Q, Liu Z, Cheng M-M, Hu S-M (2022) Segnext: rethinking convolutional attention design for semantic segmentation. *Adv Neural Inf Process Syst* 35:1140–1156
26. Li H, Li J, Wei H, Liu Z, Zhan Z, Ren Q (2024) Slim-neck by gsconv: a lightweight-design for real-time detector architectures. *J Real-Time Image Proc* 21(3):62
27. Tian D, Yan X, Zhou D, Wang C, Zhang W (2024) Iv-yolo: a lightweight dual-branch object detection network. *Sensors* 24(19):6181
28. Cheng Z, Gao L, Wang Y, Deng Z, Tao Y (2024) Ec-yolo: effectual detection model for steel strip surface defects based on yolo-v5. *IEEE Access* 12:62765–62778
29. Fu J, Chen X, Lv Z (2022) Rail fastener status detection based on mobilenet-yolov4. *Electronics* 11(22):3677
30. Li X, Wang Q, Yang X, Wang K, Zhang H (2023) Track fastener defect detection model based on improved yolov5s. *Sensors* 23(14):6457
31. Li N, Wang M, Yang G, Li B, Yuan B, Xu S (2024) Dens-yolov6: a small object detection model for garbage detection on water surface. *Multimed Tools Appl* 83(18):55751–55771
32. Wang C-Y, Bochkovskiy A, Liao H-YM (2023) Yolov7: trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp 7464–7475
33. Liu J, Liu H, Chakraborty C, Yu K, Shao X, Ma Z (2021) Cascade learning embedded vision inspection of rail fastener by using a fault detection IoT vehicle. *IEEE Internet Things J* 10(4):3006–3017
34. Wang C-Y, Liao H-YM et al (2024) Yolov1 to yolov10: the fastest and most accurate real-time object detection systems. *APSIPA Trans Signal Inf Process* 13(1):1–13
35. Wang Z, Su Y, Kang F, Wang L, Lin Y, Wu Q, Li H, Cai Z (2025) Pc-yolo11s: a lightweight and effective feature extraction method for small target image detection. *Sensors* 25(2):348

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Siwei Ma¹ · Ronghua Li² · Henan Hu¹

✉ Ronghua Li
lironghua705@163.com

Siwei Ma
ma_siwei2021@163.com

Henan Hu
huhenan18@mails.ucas.ac.cn

¹ School of Mechanical Engineering, Dalian Jiaotong University, No. 794, Yellow River Road, Dalian 116028, Liaoning, China

² School of Automation and Electrical Engineering, Dalian Jiaotong University, No. 216 Xingfa Road, Lushunkou Economic Development Zone, Dalian 116045, Liaoning, China